POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Rolling-Horizon $A^\star$ Planning and Nonlinear MPC for Map-Free Navigation of a Legged Robot

Project Report
062047 – Numerical Optimization for Control

Professor:
Prof. Lorenzo Mario Fagiano

Group Members:
Mateo Gomez
Lorenzo Ortolani
Francesco Pedrini

Academic year:
2025-2026

Abstract: Autonomous navigation in an unknown, cluttered environment can be posed as a pair of nested optimization programs solved at every control cycle: a discrete shortest-path problem on a LiDAR-derived occupancy grid, and a continuous nonlinear program (NLP) that turns the resulting geometric reference into dynamically feasible velocity commands. Here, we analyze the map-free navigation of ground legged robots, such as a Unitree Go2 dog and G1 humanoid robots. The high level plant is simple non-linear forward kinematic dyanmics, valid for each legged robot abstracted as a body-frame holonomic agent with first-order actuator lag for modeling the low level dynamics latency, yielding a six-state, three-input discrete model; the nonlinear MPC tracker formulated in CasADi and solved by IPOPT at 10 Hz. The eleven cost weights of that NLP are calibrated offline by a per-axis Bayesian scheme: one one-dimensional Gaussian process per weight, six closed-loop mission rollouts each, swept in order of influence with every optimum carried forward, for 67 rollouts in total. Every claim in the performance analysis is produced by driving the deployed CasADi/IPOPT tracker in closed loop inside an offline harness, over three synthetic worlds and eleven parametric studies. The tuned weights turn a controller that never reaches the goal into one that does, and both the per-axis curves and an independent closed-loop ablation isolate the cause: two of the eleven weights, the barrier radius and its steepness, move the mission score by a factor of 130 more than the other nine combined. The study further quantifies the cost/performance frontier of the horizon (0.52 ms per additional step, closed-loop performance saturating at $N \approx 20$), the 38.7% computational saving of the shift-and-append warm start, the conditioning penalty of a steep barrier, the mismatch-induced steady-state offset caused by the absence of integral action, and the discrepancy between the deployed heuristic terminal weight and the infinite-horizon cost-to-go obtained from the discrete algebraic Riccati equation.

Key-words: Nonlinear Model Predictive Control, Interior-Point Optimization, Rolling-Horizon Planning, Bayesian Optimization, Legged Robot Navigation

Contents

1. Introduction

A mobile robot that must reach a goal in an environment it has never seen has no global map to plan on: at every instant it knows only what its sensors reported in the last few metres. The classical answer is to decouple the problem into a geometric planner and a low-level tracker, but if the two layers ignore each other the planner produces paths the actuators cannot execute, and the tracker produces commands the geometry does not allow. Model Predictive Control (MPC) resolves the conflict by construction: it optimizes a finite sequence of future inputs subject to the actual dynamics and the actual constraints, and re-optimizes at every cycle [13]. The price is computational for the motion planner point of view: a nonlinear program must be solved to sufficient accuracy inside the sampling period, every period, on the robot. However, this rolling horizon path planning, allows the robot to avoid simultaneously localize and map the enviroment. The presence of only a localization module throguh an EKF and the lack of complete SLAM, enables less computationally expensive navigation stack, which is an important contribution of this report.
The goal is to analyze a Go2 and G1 navigation stack. The stack is deployed on a Unitree Go2 quadruped, whose locomotion layer, the CHAMP gait controller [6] on the simulated robot, the vendor's sport-mode controller on the physical one, exposes a body-frame velocity interface; the same code has since been ported to a Unitree G1 humanoid walking under the AMO reinforcement-learning policy [8], which is what makes the "robot-agnostic" claim testable, due to the simple high level holonomic kinematic model derived from both systems. Section 2.3 describes those two locomotion layers and the abstraction that makes them interchangeable. Two programs are solved at every planning cycle and are the object of the analysis:

- a *discrete* shortest-path program — an $A^\star$ search over a Gaussian occupancy grid rebuilt from the current LiDAR scan, whose solution is a geometric reference and whose rolling-horizon target is the intersection of the ray to the global goal with the grid boundary;
- a *continuous* nonlinear program — a six-state, three-input MPC with actuator-lag dynamics, a smooth obstacle barrier and box-constrained inputs, formulated once symbolically in CasADi [2] and solved by the interior-point method IPOPT [17] at 10 Hz.

On top of these two runtime programs sits a third, solved once offline: the *black-box* program of choosing the eleven cost weights of the MPC. Weight tuning is the practical bottleneck of any deployed MPC, it is not differentiable through the closed loop, and each evaluation costs a full navigation rollout. It is therefore attacked with Gaussian-process surrogates [12, 16], but *one per coordinate* rather than one over all eleven, for reasons of sample density that Section 4.4 makes precise.

2. Problem analysis

2.1. System overview

The stack is a chain of ROS 2 nodes that closes the loop between a 3-D LiDAR and the body-frame velocity interface of the locomotion controller:

1. an **odometry bridge** that converts the extended-Kalman-filter (EKF) pose estimate into a single pose topic and differentiates it, with an exponential filter, into a body-frame velocity estimate (the robot does not publish a usable body-frame twist);
2. a **mapping module** that rebuilds a local Gaussian occupancy grid from the newest scan at 1 Hz, and maintains two forms of memory, a sparse world-frame cell map with a 20 s decay and a topological navigation graph, whose purpose is precisely to repair the failure mode analysed in Section 5.3;
3. the $A^\star$ **local planner**, which solves the discrete program of Section 3.1 on that grid;
4. the **nonlinear MPC tracker**, which solves the NLP of Section 3.3 at 10 Hz and publishes a setpoint;

Two design decisions shape everything that follows. First, the locomotion layer is treated as an ideal velocity-tracking device with a time constant: the report never models legs, contacts or gait, and the robot is abstracted as a planar holonomic agent, Sections 2.2 and 2.3 describe what that layer actually does on each of the two platforms, and why the same abstraction survives the difference between them. Second, the map is *local and volatile*. The occupancy grid covers a fixed 10×10 m window centred on the robot and is rebuilt from scratch at every planning cycle, so the planning problem is genuinely map-free, and the geometric reference the MPC tracks is only valid for one grid width ahead.

2.2. The controlled platforms

The same stack drives two machines whose mechanics could hardly be more different: a Unitree Go2 quadruped (Fig. 1(a)) and a Unitree G1 humanoid (Fig. 1(b)). The Go2 has twelve actuated joints, three per leg; the

G1 has twenty-nine, of which the locomotion layer of Section 2.3.2 drives fifteen, the twelve leg joints and the three waist joints — while the arms are held at a fixed posture. Neither joint set is ever addressed by the controller of this report. Both platforms are commanded through the same channel, a body-frame twist $u = (v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, \omega^{\mathrm{cmd}})$, and both carry a 3-D LiDAR whose returns are the only exteroceptive input of the stack. Table 1 collects what distinguishes them from the controller's point of view; everything else about the two robots is, by construction, invisible to it.

(a) Unitree Go2 quadruped

photograph of the G1 humanoid
(add `Images/G1.jpeg`)

(b) Unitree G1 humanoid

Figure 1: The two platforms on which the same navigation stack is deployed. (a) The Go2 quadruped, which produced all the recorded runs of Section 5.15, with its LiDAR mounted on the head. (b) The G1 humanoid, to which the stack was ported without changing the optimization program: only the constants of the actuator-lag model (12) and the command mapping of Section 2.3.2 differ.

Table 1: The two platforms as the controller sees them. The rows above the rule are properties of the machine, those below are properties of the locomotion layer that receives the MPC command; the last row is the only one that enters the optimization program, through the prediction model (12). The two time constants were identified on the Go2 (Section 2.5) and carried over unchanged to the G1, which is the concrete form the "robot-agnostic" claim takes.

	Unitree Go2	Unitree G1
Morphology	quadruped	humanoid
Actuated joints	12 (3 per leg)	29 (15 driven by the gait policy)
Exteroception	Unitree L1 LiDAR	Livox MID-360 LiDAR
Locomotion layer	CHAMP gait controller (vendor sport mode on hardware)	AMO reinforcement-learning policy
Layer update rate	200 Hz	50 Hz
Command interface	body-frame twist u	body-frame twist u, remapped (Section 2.3.2)
Abstraction used by the MPC	first-order velocity lag, $\tau_v = 0.12$ s, $\tau_w = 0.10$ s	

2.3. Low-level locomotion layers

The controller analysed in this report never commands a joint. Its decision variable is a body-frame twist, and between that twist and the ground sits a platform-specific layer that turns it into joint commands at 200 Hz on the Go2 and 50 Hz on the G1. Only two properties of that layer reach the optimization problem: the twist is tracked with a first-order lag rather than instantaneously, and the tracking is fast enough, relative to the 10 Hz MPC cycle, for the two layers to be designed separately. This section describes the two locomotion controllers to the depth needed to justify those two properties and no further. Throughout, $q \in \mathbb{R}^{n_j}$ denotes the vector of joint positions ($n_j = 12$ on the Go2, $n_j = 29$ on the G1), $\mathcal{T} \in \mathbb{R}^{n_j}$ the vector of joint torques, and $K_P, K_D \succ 0$

the diagonal proportional and derivative gain matrices of the joint-level loop; the symbol τ is reserved for the actuator time constants of Section 2.5.

2.3.1 Model-based gait generation: CHAMP on the Go2

CHAMP [6] is an open-source implementation of the hierarchical trot controller developed for the MIT Cheetah [5, 7]. It is a purely feedforward pipeline: a pattern modulator imposes the footfall sequence, a trajectory planner turns each leg's phase into a foot position, inverse kinematics (IK) turns that position into joint angles, and a joint-level impedance loop tracks them.[1]

Pattern modulation. Let t denote time. Each leg $i \in \{\mathrm{LF}, \mathrm{RF}, \mathrm{LH}, \mathrm{RH}\}$ carries a normalised stride clock

$$\zeta_i(t) = \left(\frac{t}{T_{\mathrm{str}}} - \zeta_i^0 \right) \bmod 1 \in [0, 1), \qquad T_{\mathrm{str}} = T_{\mathrm{st}} + T_{\mathrm{sw}}, \tag{1}$$

where T_{st} and T_{sw} are the stance and swing durations and ζ_i^0 is the phase offset of leg i. Leg i is in stance while $\zeta_i < \beta$ and in swing otherwise, with duty factor $\beta = T_{\mathrm{st}}/T_{\mathrm{str}}$. The deployed configuration uses $T_{\mathrm{st}} = T_{\mathrm{sw}} = 0.25$ s, hence $T_{\mathrm{str}} = 0.5$ s and $\beta = 0.5$, and $\zeta^0 = (0, \frac{1}{2}, \frac{1}{2}, 0)$: the diagonal pairs move together, which is the trot.

Where the command enters. The commanded twist does not act on the legs directly; it acts on the *step geometry*, through the Raibert foot-placement heuristic [11]: a foot is placed half a step ahead of its nominal position, so that sweeping backwards during the stance interval T_{st} displaces the trunk by exactly the commanded amount. Let $\bar{p^{\mathrm{f}}}_i \in \mathbb{R}^3$ be the nominal (zero-command) position of foot i in the body frame. Translating the nominal stance by the half-step $\frac{1}{2}T_{\mathrm{st}}\,(v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0)^\top$ and rotating it about the vertical axis by the half-stance yaw χ_g gives the foot displacement

$$\Delta p_i^{\mathrm{f}} \;=\; R_z(\chi_g)\Big(\bar{p^{\mathrm{f}}}_i + \tfrac{1}{2}T_{\mathrm{st}}\,\big(v_x^{\mathrm{cmd}}, v_y^{\mathrm{cmd}}, 0\big)^\top\Big) - \bar{p^{\mathrm{f}}}_i, \qquad \chi_g = 2\sin\Big(\tfrac{1}{4}T_{\mathrm{st}}\,\omega^{\mathrm{cmd}}\Big) \approx \tfrac{1}{2}T_{\mathrm{st}}\,\omega^{\mathrm{cmd}}, \tag{2}$$

with $R_z(\cdot)$ the elementary rotation about the vertical axis.[2] The step length and the heading of the foot trajectory of leg i follow as

$$L_i = 2\big\|\Delta p_i^{\mathrm{f}}\big\|_2, \qquad \chi_i = \operatorname{atan2}\big(\Delta p_{i,y}^{\mathrm{f}},\, \Delta p_{i,x}^{\mathrm{f}}\big), \tag{3}$$

the factor 2 turning the half-step into the full stride of the foot. Equations (2) and (3) are the whole of the command path: the three numbers the MPC publishes are compressed into one length and one angle per leg, and nothing else of the command reaches the robot.

Foot trajectories and impedance. With phase and step length fixed, each foot follows a planar curve in its own trajectory frame, rotated by χ_i and added to the nominal stance: $p_i^{\mathrm{f}} = \bar{p^{\mathrm{f}}}_i + (\,\xi\cos\chi_i,\ \xi\sin\chi_i,\ \eta\,)^\top$, where ξ and η are the longitudinal and vertical coordinates of the curve. In stance, parametrised by the normalised stance phase $\zeta_i^{\mathrm{st}} = \zeta_i/\beta \in [0, 1)$, the foot sweeps backwards along a shallow arc that presses into the ground by h_{st},

$$\xi = \tfrac{L_i}{2}\left(1 - 2\zeta_i^{\mathrm{st}}\right), \qquad \eta = -h_{\mathrm{st}}\cos\left(\frac{\pi\,\xi}{L_i}\right); \tag{4}$$

in swing, parametrised by $\zeta_i^{\mathrm{sw}} = (\zeta_i - \beta)/(1 - \beta) \in [0, 1)$, it follows a Bézier curve of degree 11 through twelve control points $b_j \in \mathbb{R}^2$,

$$(\xi, \eta) = \sum_{j=0}^{11} \binom{11}{j} \big(\zeta_i^{\mathrm{sw}}\big)^j \big(1 - \zeta_i^{\mathrm{sw}}\big)^{11-j}\, b_j, \tag{5}$$

whose horizontal spread is scaled by L_i and whose vertical spread is scaled by the swing height h_{sw}. Figure 2 collects these quantities on a single leg. The deployed gait uses $h_{\mathrm{sw}} = 0.04$ m, $h_{\mathrm{st}} = 0.01$ m and a nominal

[1] On the physical Go2 the same body-twist interface is served by Unitree's built-in sport-mode controller, which is closed-source; CHAMP is the locomotion layer of the simulated robot, and the one whose structure can be stated. The distinction does not reach the MPC: both accept a twist and return trunk motion, and it is only that input–output behaviour that Section 2.5 models.

[2] The implementation obtains χ_g from the tangential half-step $\frac{1}{2}T_{\mathrm{st}}\,\omega^{\mathrm{cmd}} d_c$, with d_c the horizontal distance from the trunk centre to the nominal foot; d_c cancels, leaving (2). The small-angle form on the right is the yaw swept in half a stance interval, as expected.

trunk height $h_0 = 0.225$ m. A closed-form IK then maps each foot position to the three joint angles of its leg, $q_i^{\mathrm{ref}} = \mathrm{IK}_i(p_i^{\mathrm{f}})$, and a proportional–derivative (PD) law closes the loop at the joints,

$$\mathcal{T}_i = K_P\big(q_i^{\mathrm{ref}} - q_i\big) + K_D\big(\dot{q}_i^{\mathrm{ref}} - \dot{q}_i\big) \;=\; \boldsymbol{J}_i(q_i)^{\top} F_i, \tag{6}$$

where $\boldsymbol{J}_i$ is the foot Jacobian of leg i, set in bold to distinguish it from the MPC cost J and from the tuning objective $\mathcal{J}$ of Section 4.4 and F_i the equivalent force at the foot. Read from right to left, (6) is the point of the architecture: a joint-space PD loop realises a virtual spring–damper at the foot, of Cartesian stiffness $\boldsymbol{J}_i^{-\top} K_P \boldsymbol{J}_i^{-1}$, so the leg absorbs terrain irregularity and impact by programmable compliance rather than by contact sensing or force control. This is what makes an open-loop gait generator survive on a real robot.

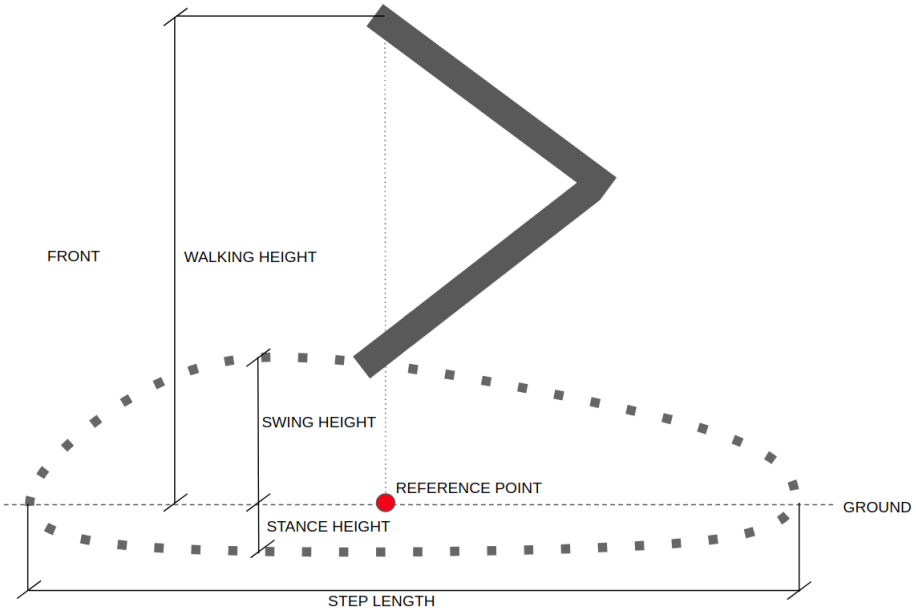

Figure 2: Geometry of the CHAMP gait on a single leg, seen from the side. The dotted closed curve is the cycle described by the foot in its trajectory frame: the upper branch is the swing Bézier (5), which clears the ground by the swing height h_{sw}, and the lower branch is the stance arc (4), which presses into the ground by the stance height h_{st} and sweeps backwards to carry the trunk forward. The horizontal extent of the cycle is the step length L_i of (3), the only channel through which the commanded twist reaches the leg; the reference point on the ground line is the nominal stance position $\bar{p}^{\mathrm{f}}_i$ about which the Raibert half-step (2) displaces the cycle, and the walking height is the nominal trunk height h_0. Adapted from the CHAMP documentation [6].

Why this layer looks like a lag. Three consequences propagate upstream, and they are the reason the model of Section 2.5 has the form it has. First, the twist enters only through the step geometry (3): no measured trunk velocity is fed back, so the layer is a feedforward map and the trunk velocity approaches the commanded one over roughly one stride rather than immediately. Second, that step geometry is re-evaluated only once per stride, so a command that changes faster than $1/T_{\mathrm{str}} = 2$ Hz is averaged, not tracked; a first-order lag with $\tau_v = 0.12$ s reaches 98% of its final value in $4\tau_v \approx T_{\mathrm{str}}$, which is why the one-parameter fit of Section 2.5 describes the layer as well as it does. Third, the gait applies its own saturation, $|v_x^{\mathrm{cmd}}| \le 0.3$ m/s, $|v_y^{\mathrm{cmd}}| \le 0.25$ m/s and $|\omega^{\mathrm{cmd}}| \le 0.5$ rad/s in the deployed configuration.

2.3.2 Learned whole-body control: the AMO policy on the G1

On the humanoid the same interface is produced by a neural network instead of a gait generator. The deployed controller is AMO (Adaptive Motion Optimization) [8], a reinforcement-learning (RL) policy trained in simulation for the G1 and its twenty-nine degrees of freedom (DoF). AMO is deliberately *not* a walking controller: it is a loco-manipulation policy, trained to track torso height, torso orientation and upper-body targets while walking, on a hybrid dataset in which an offline trajectory-optimization module supplies whole-body references that are kinematically consistent with the commanded end-effector motion. The navigation stack exercises only its locomotion subset, the arms are held at the default posture and only the base command is driven, but the policy is the one trained for the larger task, which is what makes the same platform usable for manipulation later without changing the layer the MPC talks to.

Structure. At each tick k of the 50 Hz policy loop, the network maps an observation to an action,

$$a_k = \pi(o_k) \in \mathbb{R}^{15}, \qquad o_k = \big(\, \omega_k^{\mathrm{b}},\ \hat{g}_k,\ \tilde{u}_k,\ q_k - q^{\mathrm{def}},\ \dot{q}_k,\ a_{k-1} \,\big), \tag{7}$$

where $\omega_k^{\mathrm{b}} \in \mathbb{R}^3$ is the body angular velocity, $\hat{g}_k \in \mathbb{R}^3$ the gravity direction expressed in the body frame (the tilt an inertial measurement unit (IMU) provides without a global heading), $q_k, \dot{q}_k$ the measured joint positions and velocities of the 23 observed joints, q^{def} the default posture, a_{k-1} the previous action, and $\tilde{u}_k$ the command. The action is not a torque: it is an offset on the default posture, which the same joint-level PD law as (6) converts into torque,

$$q_k^{\mathrm{ref}} = q^{\mathrm{def}} + s_a\, a_k, \qquad \mathcal{T}_k = K_P\big(q_k^{\mathrm{ref}} - q_k\big) + K_D\big(0 - \dot{q}_k\big), \tag{8}$$

with s_a a fixed action scale and a zero reference joint velocity. The learned part of the controller is therefore exactly the map from observation to PD setpoint; the stabilising loop underneath it is the same fixed impedance as on the quadruped. The policy π is obtained by proximal policy optimization (PPO) [15], maximising the expected discounted return $\mathbb{E}\big[\sum_k \gamma^k \mathcal{R}_k\big]$ with $\gamma \in (0,1)$ and a stage reward $\mathcal{R}_k$ that pays for tracking the commanded twist and penalises posture deviation, joint effort and action rate; masses, friction, gains and latencies are randomised during training so that the fixed policy transfers to hardware [14].

2.3.3 The cascade and its abstraction seam

Fig. 3 places the two locomotion layers in the control structure they belong to. The stack is a cascade of four loops whose rates are separated by roughly a decade each: the $A^\star$ reference is recomputed at 1 Hz, the MPC at 10 Hz, the locomotion layer at 50–200 Hz, and the joint PD loop at the motor-driver rate. That separation is the standard justification for designing each layer against a static abstraction of the one below it, and the abstraction used here is deliberately minimal: three decoupled first-order lags, Eq. (12), whose time constants are identified in Section 2.5. Both locomotion layers, the hand-designed one and the learned one, are approximated by the same two numbers.

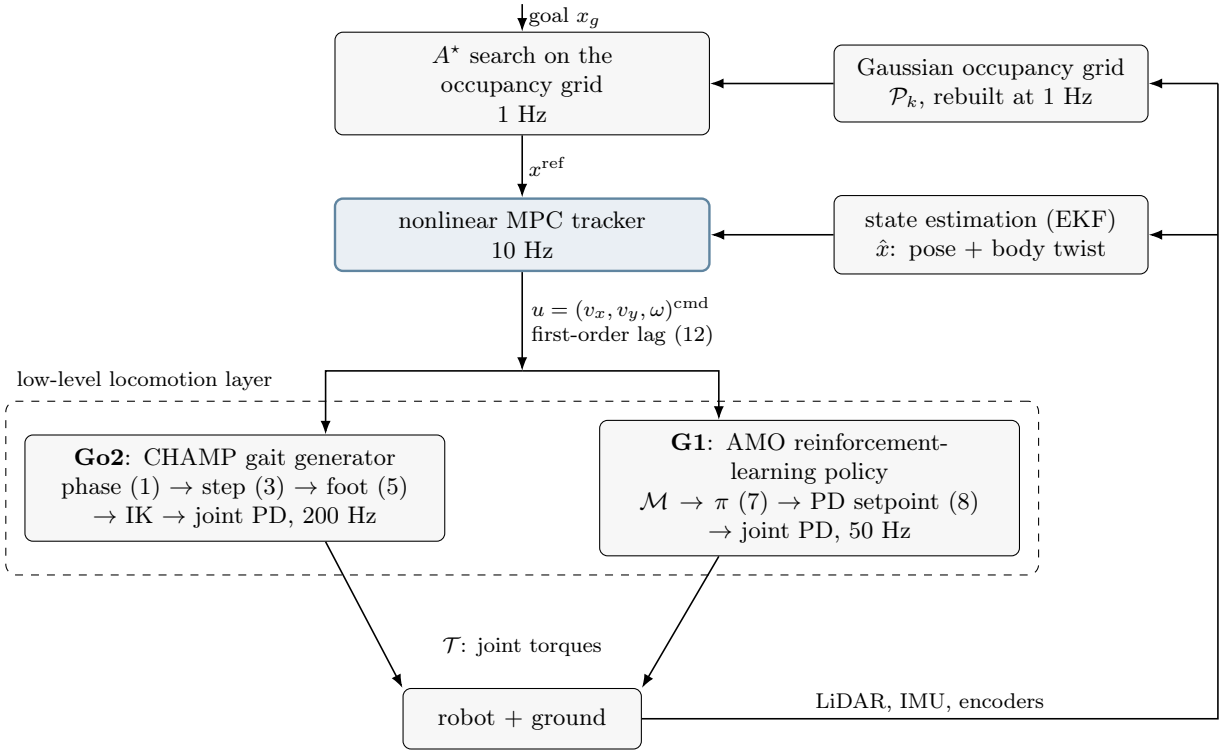

Figure 3: Cascaded structure of the controller, from the goal down to joint torques. The report analyses the two stages at the top; everything inside the dashed box is a platform-specific locomotion layer that the MPC sees only through the first-order lag (12), which is the abstraction seam of the design. The rates fall by roughly a decade at every seam (1 Hz, 10 Hz, 50–200 Hz, motor-driver rate), which is what allows the layers to be designed separately, and the two branches inside the dashed box, a hand-designed gait generator and a learned policy, are interchangeable precisely because they present the same twist interface u.

2.4. Environment representation

Let $\mathcal{P}_k = \{p_i \in \mathbb{R}^3\}_{i=1}^{N_k}$ be the LiDAR point cloud available at cycle k, projected on the plane. The grid is a square of side $2h_w$ ($h_w = 5$ m) centred on the robot position, discretised with resolution $\Delta = 0.25$ m into $n_c = 2h_w/\Delta = 40$ cells per axis. The occupancy probability of cell c is a Gaussian tail of the distance to the nearest return,

$$P(c) \;=\; 1 - \Phi\!\left(\frac{d_{\min}(c,\mathcal{P}_k)}{\sigma}\right), \qquad d_{\min}(c,\mathcal{P}_k) = \min_{p\in\mathcal{P}_k} \|c-p\|_2, \tag{9}$$

with Φ the standard normal cumulative distribution function,

$$\Phi(x) \;=\; \frac{1}{\sqrt{2\pi}}\int_{-\infty}^{x} e^{-t^2/2}\,\mathrm{d}t \;=\; \frac{1}{2}\left(1+\operatorname{erf}\!\left(\frac{x}{\sqrt{2}}\right)\right) \in (0,1), \tag{10}$$

and $\sigma = 0.15$ m. Equation (9) is an *inflation* model: it is not a Bayesian occupancy update (there is no recursive filtering, no ray casting and no free-space carving), but a smooth, monotone map from clearance to cost, whose one parameter σ sets the width of the obstacle halo. Cells with $P(c) \geq P_{\text{thr}} = 0.4$ are hard obstacles; note that $1-\Phi(1) \approx 0.159$ and $1-\Phi(0.25) \approx 0.40$, so the hard boundary sits at $d_{\min} \approx 0.25\sigma = 0.0375$ m of a return, the effective body clearance is produced by the *soft* cost of Section 3.1, not by the threshold. The left panel of Fig. 4 shows the trade-off governed by σ: a small σ leaves doorways open but hugs walls, a large one closes gaps the robot could physically cross.

Because the grid is rebuilt from scratch, its construction cost is the dominant cost of the planning layer, $O(n_c^2\,|\mathcal{P}_k|)$; vectorised over 40×40 cells and $\sim$180 returns it measures 4–5 ms on average together with the $A^\star$ search (Section 5.1), which at the 1 Hz replanning rate is negligible against the 10 Hz MPC.

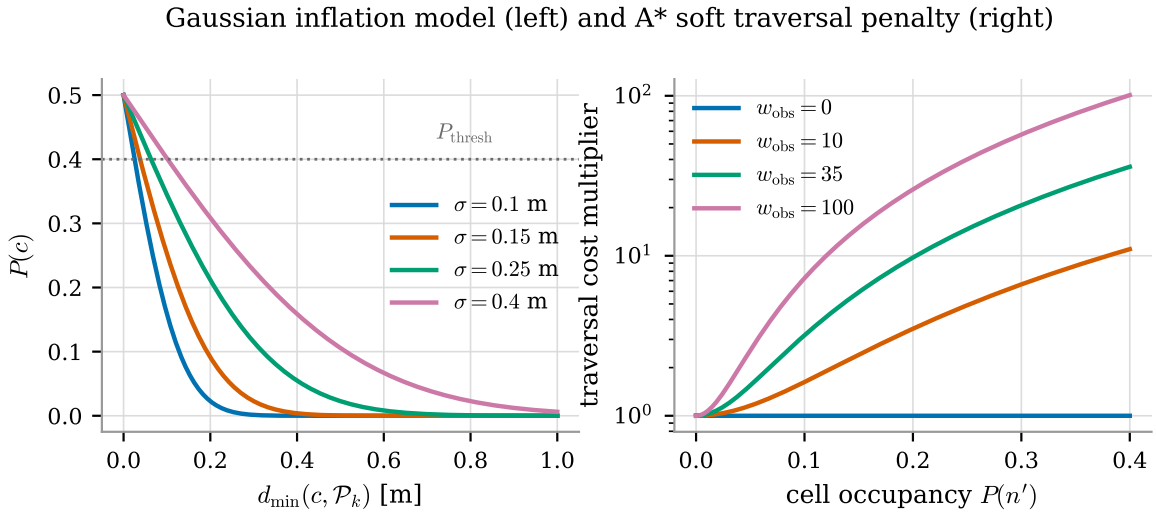

Figure 4: Left: the Gaussian inflation model (9) for four values of σ, with the hard threshold $P_{\text{thr}} = 0.4$. Right: the soft traversal-cost multiplier $1 + w_{\text{obs}}(P/P_{\text{thr}})^2$ of the $A^\star$ edge cost (15), which is what actually keeps the path away from walls; the deployed value is $w_{\text{obs}} = 35$.

2.5. Plant model and state-space formulation

The MPC model must predict where the robot will be, not where it was told to go. Commanding a legged platform a body-frame velocity does not produce that velocity instantaneously: the locomotion controller responds with a measured time constant of $\tau \approx 0.10$–0.15 s, which over a 5 s horizon accumulates into a prediction error of several tens of centimetres if neglected. The model therefore augments the usual planar kinematic state with the three velocity states, giving

$$x \;=\; \begin{bmatrix} p_x & p_y & \psi & v_x & v_y & \omega \end{bmatrix}^\top \in \mathbb{R}^6, \qquad u \;=\; \begin{bmatrix} v_x^{\text{cmd}} & v_y^{\text{cmd}} & \omega^{\text{cmd}} \end{bmatrix}^\top \in \mathbb{R}^3, \tag{11}$$

where (p_x, p_y) and ψ are the world-frame pose and (v_x, v_y, ω) the body-frame twist actually realised by the platform. In continuous time the actuator behaves as three decoupled first-order lags driven by the commands,

$$\dot{v}_x = \frac{u_1 - v_x}{\tau_v}, \qquad \dot{v}_y = \frac{u_2 - v_y}{\tau_w}, \qquad \dot{\omega} = \frac{u_3 - \omega}{\tau_w}, \tag{12}$$

while the pose obeys the holonomic body-to-world rotation

$$\dot{p}_x = v_x \cos\psi - v_y \sin\psi, \qquad \dot{p}_y = v_x \sin\psi + v_y \cos\psi, \qquad \dot{\psi} = \omega. \tag{13}$$

The identified constants are $\tau_v = 0.12$ s and $\tau_w = 0.10$ s.[3]
Model (12)–(13) is holonomic: unlike a differential-drive vehicle, the platform can command a lateral velocity, so no nonholonomic constraint appears and the input set is a box. This is what makes the resulting NLP comparatively benign — the nonlinearity is confined to the rotation $R(\psi)$ and to the obstacle distances.
Fig. 5 quantifies why the lag states are worth their computational cost. The left panel compares the true velocity response with the instantaneous model $v \equiv u$ for a step command; the right panel integrates the resulting velocity discrepancy over one 5 s horizon and reports the peak position error as a function of τ_v. At the identified $\tau_v = 0.12$ s the instantaneous model mispredicts the horizon end-point by ∼7 cm, which is of the same order as the tracking accuracy the controller is asked to deliver.

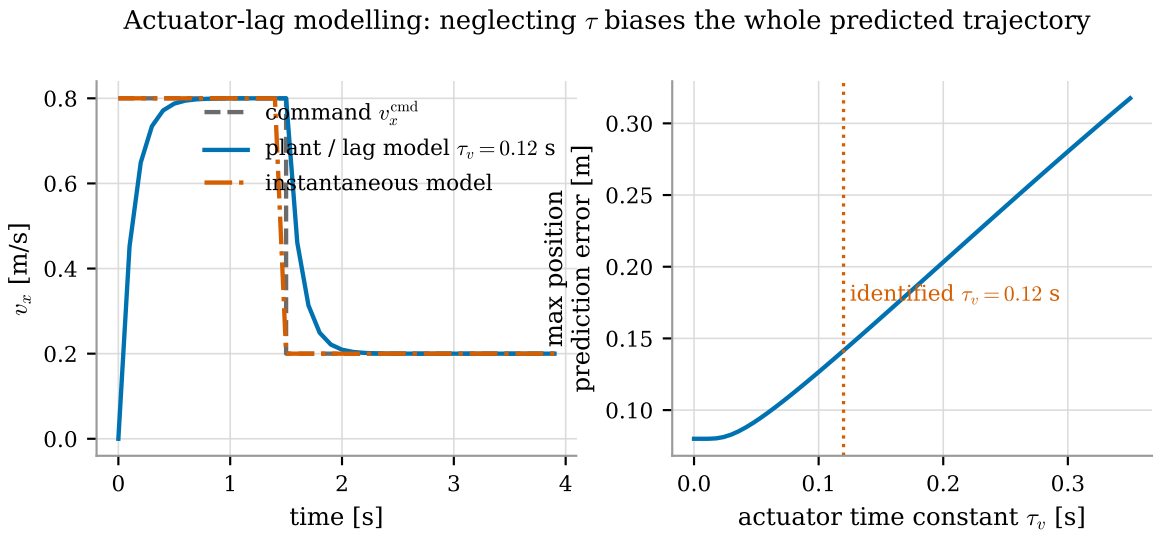

Figure 5: Actuator-lag modelling. Left: response of the first-order-lag model (12) against the instantaneous model $v \equiv u$ for a step in commanded forward velocity. Right: maximum position prediction error accumulated over one 5 s horizon by neglecting the lag, as a function of τ_v.

2.6. Navigation problem statement

Let $x_g \in \mathbb{R}^2$ be the goal. The navigation task is the receding-horizon optimal control problem

$$\begin{aligned}
\min_{u_0,\dots,u_{N-1}} \quad & J(x_{0:N}, u_{0:N-1}, \mathcal{P}_k, \theta) && && \text{(14a)}\\
\text{s.t.} \quad & x_{t+1} = f(x_t, u_t), && t = 0,\dots,N-1, && \text{(14b)}\\
& x_0 = \hat{x}(t_k), && && \text{(14c)}\\
& u_t \in \mathcal{U}, && t = 0,\dots,N-1, && \text{(14d)}\\
& d(x_t, \mathcal{P}_k) \geq r_{\min}, && t = 0,\dots,N, && \text{(14e)}
\end{aligned}$$

solved at every sampling instant t_k, of which only u_0 is applied. Three features of (14) determine the whole design:

- **The goal is not reachable within the horizon.** With $N\Delta t = 5$ s and a cruise speed of 0.5 m/s the horizon spans 2.5 m, while the goal may be tens of metres away and outside the sensed region altogether. The cost (14a) therefore cannot be a distance to x_g; it must be a distance to a *reference trajectory* produced by a separate, cheaper program that looks further ahead. That program is the $A^\star$ search of Section 3.1.

[3] In the deployed code the lateral velocity v_y is propagated with τ_w rather than τ_v, i.e. lateral motion is assumed to respond as fast as yaw. The two constants differ by 20 ms, well inside the identification uncertainty, so the substitution is numerically immaterial; it is reported here because the analysis reproduces the deployed formulation exactly rather than an idealisation of it.

- **The obstacle set is a point cloud, not a polytope.** Constraint (14e) is non-convex, and its description changes at every cycle. It is relaxed into a smooth penalty (Section 3.3.2), which keeps the NLP smooth and always feasible at the price of losing the hard safety guarantee.
- **The cost is parametrised by θ.** The eleven weights in θ are not physical quantities; they are design variables of a second-level optimization problem (Section 4.4).

3. Formulation of the optimization program

3.1. Reference generation: $A^\star$ as a discrete program

At every planning tick the geometric reference is the solution of a shortest-path problem on the 8-connected grid of Section 2.4. Cells with $P \geq P_{\text{thr}}$ are removed from the graph; the remaining edges carry the cost

$$g(n \to n') \;=\; c_{\text{move}} \cdot \Delta \cdot \left[1 + w_{\text{obs}} \left(\frac{P(n')}{P_{\text{thr}}}\right)^2\right], \qquad c_{\text{move}} \in \{1, \sqrt{2}\}, \tag{15}$$

with $w_{\text{obs}} = 35$. The normalised quadratic in (15) is the mechanism that produces medial-axis-like paths: a cell just below the hard threshold costs $36\times$ an open cell, so the search prefers the middle of a corridor over the short path that grazes a wall, yet a doorway is never closed outright. The heuristic

$$h(n) \;=\; \Delta\sqrt{(i_x - g_x)^2 + (i_y - g_y)^2} \tag{16}$$

is admissible because the bracket in (15) is bounded below by 1, so no edge can ever be cheaper than its Euclidean length; the search is therefore an exact $A^\star$ and returns a cost-optimal path on the current grid. Diagonal expansions are additionally rejected when either of the two orthogonal neighbours is blocked, which prevents the path from squeezing through a gap of zero width that the physical body cannot cross.

Rolling horizon. Because the grid is local, the $A^\star$ target is

$$x_{\text{local}} \;=\; \begin{cases} \text{cell}(x_g), & x_g \in \text{grid}, \\ \text{ray}(x_0 \to x_g) \cap \partial\,\text{grid}, & \text{otherwise}, \end{cases} \tag{17}$$

i.e. when the goal is out of the window the planner aims at the boundary cell along the ray to the goal, advancing one grid width at a time. If the selected cell is occupied, a breadth-first search returns the nearest free cell, so $A^\star$ always receives a valid target. The resulting grid path is resampled at $\Delta s = 0.15$ m and smoothed with a 7-tap moving average (endpoints preserved) to remove the grid-induced zig-zag before it is handed to the MPC. Equation (17) is a greedy, purely reactive rule, and it is the single most consequential approximation in the stack: the ray to the goal carries no information about the free-space topology, so a wall that separates the robot from the goal is re-discovered at every cycle and never remembered. Section 5.3 shows the resulting livelock in a two-room world, which is exactly what the persistent cell map and the navigation graph of Section 2.1 exist to prevent.

3.2. Model discretization

With sampling time Δt, the lag subsystem (12) is linear and time-invariant, so it is discretised *exactly* under a zero-order-hold input. Defining

$$\lambda_v = 1 - e^{-\Delta t/\tau_v}, \qquad \lambda_w = 1 - e^{-\Delta t/\tau_w}, \tag{18}$$

the velocity update is the exact solution of (12) over one step,

$$v_{x,k+1} = (1-\lambda_v)v_{x,k} + \lambda_v u_{1,k}, \qquad v_{y,k+1} = (1-\lambda_w)v_{y,k} + \lambda_w u_{2,k}, \qquad \omega_{k+1} = (1-\lambda_w)\omega_k + \lambda_w u_{3,k}. \tag{19}$$

The pose is then integrated with one forward-Euler step, but evaluated on the *post-lag* velocity:

$$\begin{aligned} p_{x,k+1} &= p_{x,k} + (v_{x,k+1}\cos\psi_k - v_{y,k+1}\sin\psi_k)\,\Delta t, \\ p_{y,k+1} &= p_{y,k} + (v_{x,k+1}\sin\psi_k + v_{y,k+1}\cos\psi_k)\,\Delta t, \\ \psi_{k+1} &= \psi_k + \omega_{k+1}\Delta t. \end{aligned} \tag{20}$$

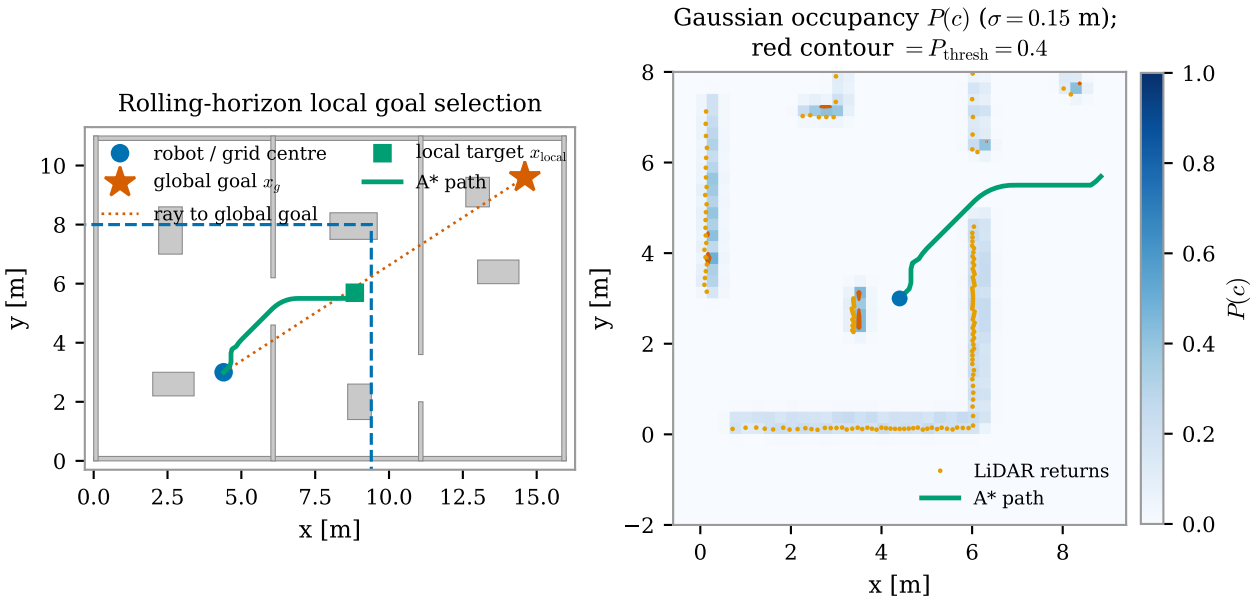

Figure 6: Left: rolling-horizon target selection (17) — the dashed square is the 10×10 m local grid, the dotted line the ray to the global goal, the square marker the local target where that ray meets the grid boundary, and the solid line the $A^\star$ solution. Right: the Gaussian occupancy field (9) built from the current returns, with the $P = 0.4$ hard-obstacle contour.

Using v_{k+1} instead of v_k in (20) is a semi-implicit (symplectic-Euler) choice: it costs nothing, removes the one-step delay that explicit Euler would introduce between a command and the predicted displacement, and makes the discrete model reproduce the measured step response of the platform. Equations (19)–(20) define the map $x_{k+1} = f(x_k, u_k)$ used in (14b). Only $\cos\psi_k$, $\sin\psi_k$ and the obstacle distances are nonlinear; f is C^∞ and its Jacobian is sparse and analytically available through CasADi's algorithmic differentiation.

3.3. The nonlinear MPC program

3.3.1 Decision variables and cost

The NLP is written in the *simultaneous* (direct multiple-shooting with a full state parametrisation) form: both the state and the input sequences are decision variables and the dynamics enter as equality constraints,

$$z \;=\; \big(X, U\big), \qquad X = [x_0, \ldots, x_N] \in \mathbb{R}^{6\times(N+1)}, \quad U = [u_0, \ldots, u_{N-1}] \in \mathbb{R}^{3\times N}. \tag{21}$$

This is the standard choice for interior-point solvers: it keeps the Jacobian sparse and banded, whereas a condensed single-shooting formulation would produce a small but dense problem with a much worse conditioned Hessian over a 50-step horizon.
With $e_k = x_k - x_{\text{ref},k}$ the cost is

$$J(z;\theta) \;=\; \underbrace{\sum_{k=0}^{N-1} \Big[e_k^\top Q\, e_k \;+\; u_k^\top R\, u_k \;+\; R_{\text{jerk}} \|u_k - u_{k-1}\|_2^2 \;+\; J_{\text{obs}}(x_k) \Big]}_{\text{stage cost}} \;+\; \underbrace{e_N^\top Q_T\, e_N + J_{\text{obs}}(x_N)}_{\text{terminal cost}}, \tag{22}$$

with

$$Q = \operatorname{diag}(Q_x,\, Q_y,\, Q_\psi,\, 0,\, 0,\, 0), \qquad Q_T = \rho\, Q, \qquad R = \operatorname{diag}(R_{v_x}, R_{v_y}, R_\omega). \tag{23}$$

Three properties of (22)–(23) deserve comment.
The stage weight is only semi-definite. No velocity state is penalised: Q has rank 3 out of 6. The velocities are shaped indirectly, through the lag dynamics and through the input penalty R. This keeps the reference generator simple — an $A^\star$ path carries no velocity information — but it removes any direct damping of the velocity states from the cost, and it is the reason why the Riccati comparison of Section 3.3.4 requires a regularisation.
The rate penalty couples consecutive inputs. The term $R_{\text{jerk}}\|u_k - u_{k-1}\|^2$ (with u_{-1} the previously applied command) is the discrete analogue of an input-rate penalty. It is the only term that introduces off-diagonal

blocks between adjacent input stages in the Hessian, and its purpose is physical: a legged platform tracks a smooth velocity profile far better than a chattering one.
The terminal cost is a scaled copy of the stage cost. $Q_T = \rho Q$ with $\rho = Q_{\text{terminal}}$ is a heuristic, not a Lyapunov function: it is neither the solution of a Riccati equation nor an exact cost-to-go, and it acts only on the three position/heading components. Section 3.3.4 quantifies the gap.

3.3.2 Obstacle avoidance as a smooth barrier

The hard clearance constraint (14e) is replaced by a penalty evaluated on the M nearest LiDAR returns $\{p_j\}_{j=1}^{M}$ inside a search radius of 3 m:

$$J_{\text{obs}}(x_k) \;=\; W_{\text{obs}} \sum_{j=1}^{M} \left[\underbrace{\tfrac{1}{2}\Big(1 - \tanh\big(\tfrac{\alpha_{\text{obs}}}{2}(d_{k,j} - r_{\text{obs}})\big)\Big)}_{\text{soft sigmoid zone}} + \underbrace{2\max\big(0,\; r_{\text{obs}} - d_{k,j}\big)^2}_{\text{quadratic penetration penalty}} \right], \tag{24}$$

$$d_{k,j} = \sqrt{(p_{x,k} - p_{j,x})^2 + (p_{y,k} - p_{j,y})^2 + \varepsilon}, \qquad \varepsilon = 10^{-6}. \tag{25}$$

The additive constant ε makes (25) differentiable at zero distance, where the Euclidean norm is not. The logistic term is written with tanh rather than as $1/(1 + e^{\alpha_{\text{obs}}(d - r_{\text{obs}})})$ because the two are algebraically identical while the former does not overflow for large arguments.
The hybrid structure of (24) is deliberate and is best read from Fig. 7. The sigmoid saturates at W_{obs} as $d \to 0$: on its own it is a *bounded* penalty, so a sufficiently strong tracking term can always buy its way through an obstacle, and — worse for the solver — its gradient vanishes both far away and deep inside the obstacle, leaving a stationary point precisely where the robot must be pushed out. The quadratic term $2\max(0, r_{\text{obs}} - d)^2$ repairs both defects inside the safety radius: it is unbounded and its gradient grows linearly with penetration. The price is visible in the third panel: the curvature of (24) scales as $\alpha_{\text{obs}}^2 W_{\text{obs}}$, so a steep, heavy barrier injects large, rapidly varying eigenvalues into the Hessian exactly in the region where the solution lives. This is the conditioning trade-off measured in Section 5.10.

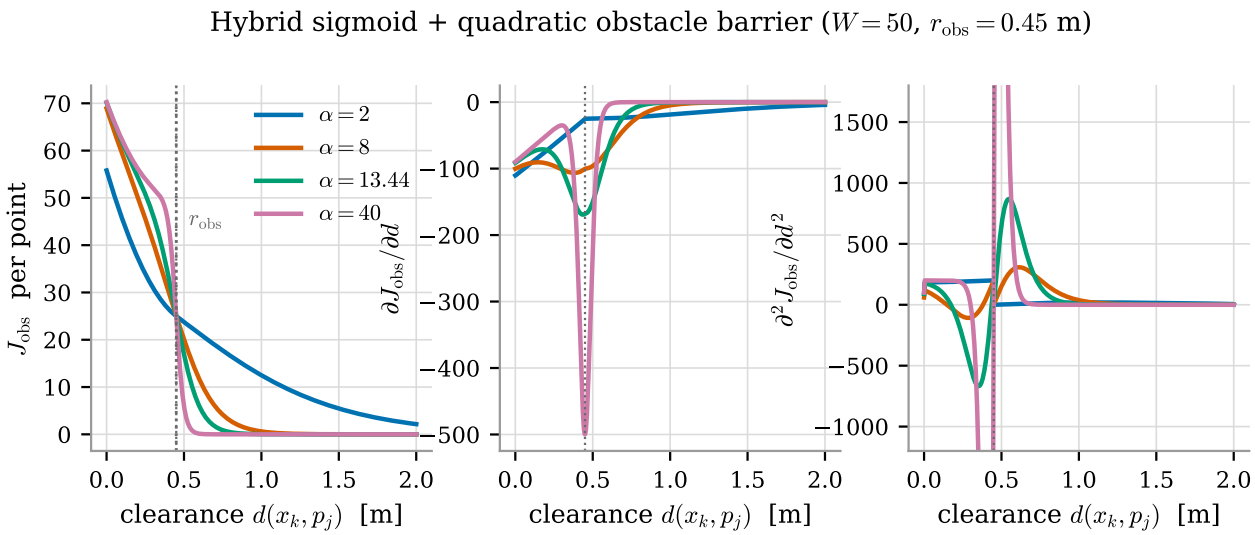

Figure 7: The hybrid barrier (24) for four steepness values ($W_{\text{obs}} = 50$, $r_{\text{obs}} = 0.45$ m): value (left), gradient (centre) and curvature (right) versus clearance. The sigmoid alone saturates and loses its gradient inside the obstacle; the quadratic term restores an unbounded, linearly growing restoring force for $d < r_{\text{obs}}$. The curvature grows as α_{obs}^2, which is what degrades the conditioning of the NLP.

Relaxing a hard constraint into a penalty is a modelling choice with three consequences worth stating explicitly. *(i)* The NLP is always feasible — there is no state constraint that can render the problem infeasible when the robot is already too close to an obstacle, which for a real-time loop with no fallback trajectory is a significant robustness advantage. *(ii)* Safety is no longer guaranteed: (24) is not an exact penalty, so a finite W_{obs} admits a finite violation. A control-barrier formulation [1] would recover the guarantee at the price of feasibility and of a much harder NLP. *(iii)* The barrier is evaluated only at the $N + 1$ discrete nodes, so nothing forbids the continuous inter-sample trajectory from clipping a thin obstacle — a further reason to keep Δt small (Section 5.9).

Sentinel padding. $M = 12$ is fixed. When fewer than M returns lie inside the search radius, the missing rows of the obstacle parameter are filled with a sentinel at 10^3 m, whose barrier contribution is numerically zero. This apparently cosmetic detail is what allows the NLP to be built *once*: the number of terms in (24), and therefore the sparsity pattern of the Jacobian and Hessian, never changes between cycles, so the symbolic factorisation and the warm start remain valid regardless of how cluttered the scene is.

3.3.3 Constraints

The program carries only equality constraints and input bounds:

$$x_{k+1} - f(x_k, u_k) = 0, \qquad k = 0, \dots, N-1, \tag{26a}$$
$$x_0 - \hat{x} = 0, \tag{26b}$$
$$0 \;\le\; u_{1,k} \;\le\; v_{x,\max}, \qquad k = 0, \dots, N-1, \tag{26c}$$
$$|u_{2,k}| \le v_{y,\max}, \qquad |u_{3,k}| \le \omega_{\max}, \qquad k = 0, \dots, N-1, \tag{26d}$$

with $v_{x,\max} = 1.0$ m/s, $v_{y,\max} = 0.5$ m/s and $\omega_{\max} = 1.5$ rad/s. Three remarks:

- **No state constraints.** Neither the obstacle clearance nor the velocity states are constrained; the former is penalised, the latter are bounded implicitly because (19) is a convex combination of a bounded command and the previous velocity, so $|v_{x,k}| \le v_{x,\max}$ for all k whenever $|v_{x,0}| \le v_{x,\max}$. The feasible set of the NLP is therefore a box intersected with an affine-in-X manifold, and a feasible point is always trivially available (integrate any admissible input sequence).
- **Forward velocity is non-negative.** Constraint (26c) forbids reversing. This is a deployment decision — the LiDAR field of view and the gait controller both favour forward motion — but it removes the manoeuvre that would resolve a dead end, and it therefore contributes to the livelock of Section 5.3.
- **Bounds are parameters, not constants.** $v_{x,\max}$, $v_{y,\max}$ and $\omega_{\max}$ enter the NLP as CasADi parameters, so the supervisor can shrink them at runtime (Section 4.3) without rebuilding the problem.

3.3.4 Terminal cost, stability and recursive feasibility

The formulation carries no terminal set and no terminal constraint, so none of the standard guarantees of nominal stability and recursive feasibility [3, 9] applies. Recursive feasibility is nevertheless *structurally* present, for the reason given above: with no state constraints, every input sequence in the box is feasible, so the NLP always has a solution and the closed loop can never be aborted by infeasibility. What is lost is the guarantee that the closed loop converges: the tracking cost may decrease along a trajectory that walks into a barrier well, and only the penalty term prevents that.

It is instructive to compare the deployed terminal weight with the exact infinite-horizon cost-to-go of the *linearised* problem. Linearising (19)–(20) about straight-line cruise at $v_{\text{ref}} = 0.5$ m/s and $\psi = 0$ gives a time-invariant pair (A, B), and with the deployed Q, R of (23) the discrete algebraic Riccati equation

$$P_\infty = A^\top P_\infty A - A^\top P_\infty B \left(R + B^\top P_\infty B\right)^{-1} B^\top P_\infty A + Q \tag{27}$$

yields the optimal cost-to-go $V_\infty(e) = e^\top P_\infty e$ and gain $K_\infty = -(R + B^\top P_\infty B)^{-1} B^\top P_\infty A$. Because Q in (23) is only semi-definite, (27) is solved on $Q + 10^{-9} I$. The closed-loop spectral radius is $\rho(A + BK_\infty) = 0.953$, i.e. the linearised cruise dynamics under the infinite-horizon controller are (marginally) stable; Table 2 compares $\operatorname{diag}(P_\infty)$ with the deployed $\operatorname{diag}(\rho Q)$.

Table 2: Deployed heuristic terminal weight $Q_T = \rho Q$ ($\rho = 184.78$) against the infinite-horizon cost-to-go P_∞ from the DARE (27) of the linearised cruise dynamics. The last row is the multiplier $\rho_i = P_{\infty,ii}/Q_{ii}$ that would reproduce P_∞ component-wise.

	p_x	p_y	ψ	v_x	v_y	ω
$\operatorname{diag}(P_\infty)$	378.4	325.3	49.4	1.31	0.66	0.15
$\operatorname{diag}(\rho Q)$	15042.4	14190.0	29.5	0	0	0
implied ρ_i	4.65	4.24	308.7	—	—	—

The deployed terminal cost over-weights terminal position error by a factor $\sim$40 with respect to the true cost-to-go, under-weights heading by $\sim$1.7, and ignores the terminal velocity states and the P_∞ cross-couplings ($P_{\infty,12} = 42.4$ between p_y and ψ, $P_{\infty,03} = 19.3$ between p_x and v_x) entirely. Fig. 8 visualises the discrepancy. That the closed loop is nevertheless almost insensitive to ρ (Section 5.13) is not a contradiction: with $N\Delta t = 5$ s the horizon already outlives the transient, and the reference is regenerated every second, so the terminal term is a small correction rather than the stabilising ingredient it is in a short-horizon design.

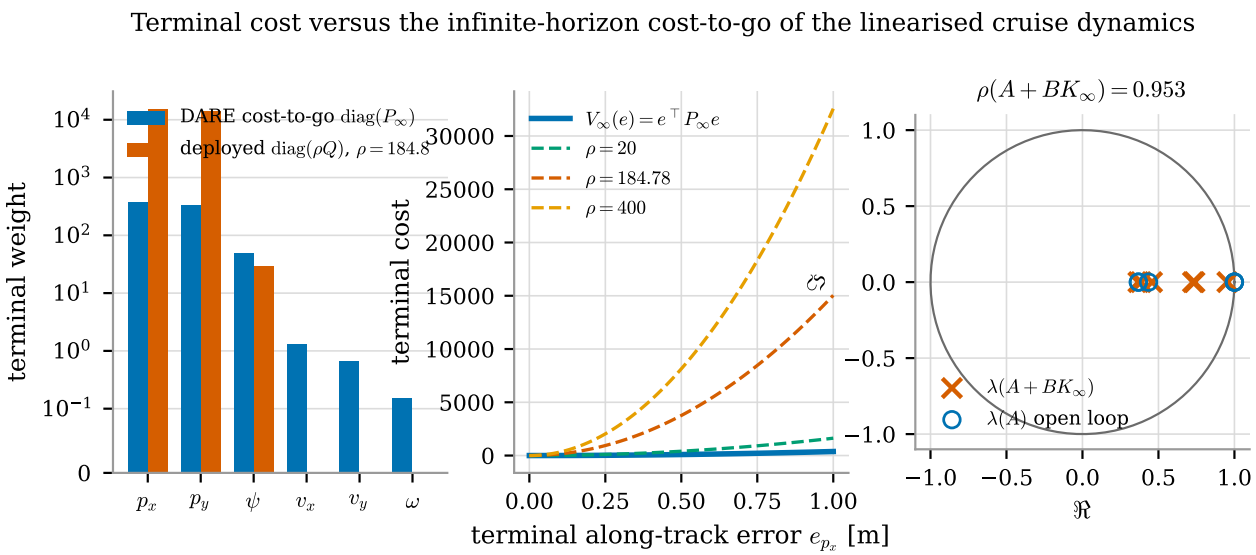

Figure 8: Terminal cost versus the infinite-horizon cost-to-go of the linearised cruise dynamics. Left: diagonal of P_∞ against the deployed ρQ (symmetric-log scale). Centre: terminal cost along a pure along-track error for V_∞ and for three values of ρ. Right: open-loop eigenvalues of A and closed-loop eigenvalues of $A + BK_\infty$ in the unit disc.

3.3.5 Reference trajectory and warm start

The reference $\{x_{\text{ref},k}\}_{k=0}^{N}$ is obtained by advancing along the smoothed $A^\star$ path at the cruise speed $v_{\text{ref}} = 0.5$ m/s from the arc-length coordinate s_0 of the path point closest to the robot,

$$s_k = \min(s_0 + v_{\text{ref}}\, k\, \Delta t,\ L_{\text{path}})\,, \tag{28}$$

linearly interpolating the position inside the containing segment and taking the heading as the segment tangent, $\psi_{\text{ref},k} = \operatorname{atan2}(\Delta y, \Delta x)$. When the path is shorter than $v_{\text{ref}} N \Delta t$ the reference saturates at its end point, which turns the tracking term into a terminal-position term — the mechanism by which the controller decelerates into the local goal.

The previous solution is reused as the initial guess by the standard shift-and-append rule

$$U_{\text{new}}^{(0)} = \left[u_1^\star, \ldots, u_{N-1}^\star, u_{N-1}^\star\right], \qquad X_{\text{new}}^{(0)} = \left[x_1^\star, \ldots, x_N^\star, x_N^\star\right], \tag{29}$$

which is consistent to first order because the reference itself shifts by one step. Two safeguards protect the warm start: it is discarded whenever IPOPT reports a failure, and it is discarded when the optimal cost exceeds five times the mean of the last seven solves. The second rule matters because a cost spike signals that the reference topology has jumped (a new wall entered the window, the $A^\star$ path switched homotopy class), in which case the shifted guess lies in the basin of the *old* solution and would slow the solver down or drag it to a poor local minimum. Section 5.8 quantifies the benefit of the warm start at 38.7% of the computation time.

3.3.6 On the absence of integral action

The formulation has no integral action and no disturbance estimator: the state (11) is not augmented with an input-disturbance model, and the reference (28) is tracked in a purely proportional–predictive sense. For a nominal plant this is immaterial, but any constant unmodelled input — a mis-identified τ, a lateral drift of the gait, a slope — produces a persistent offset from the path rather than being integrated away. Section 5.14 measures that offset; Section 6 discusses the standard remedy (state augmentation with an integrated tracking error, as in the velocity-form MPC of [13]).

4. Optimization procedure

4.1. Problem dimensions and structure

Table 3 reports the size of the NLP at the deployed setting $N = 50$, $M = 12$. It is a medium-scale, sparse, smooth, non-convex problem: 456 decision variables, 306 equality constraints, 300 simple bounds and 1224 barrier terms in the objective. The Jacobian of (26a) is block bi-diagonal, with 6×9 blocks coupling (x_k, u_k) to x_{k+1}; the Hessian of the Lagrangian is block tri-diagonal, the off-diagonal input blocks coming from the rate penalty. Nothing in the problem couples distant stages, so the linear algebra cost of an interior-point iteration is linear in N — a prediction confirmed to within a few percent by the measured 0.52 ms per additional horizon step (Section 5.7).

Table 3: NLP dimensions at the deployed configuration $N = 50$, $\Delta t = 0.1$ s, $M = 12$.

Quantity	Expression	Value
State variables	$6(N+1)$	306
Input variables	$3N$	150
Total decision variables	$6(N+1)+3N$	**456**
Dynamic equalities	$6N$	300
Initial-state equalities	6	6
Input bounds	$6N$	300
Barrier terms (sigmoid + quadratic)	$2M(N+1)$	1224
Nonlinear elementary functions per solve	$2(N+1)M+2N$	1324

4.2. Solver methodology

The NLP is solved with IPOPT [17], a primal–dual interior-point method with a filter line-search. Writing the problem as $\min_z \phi(z)$ s.t. $c(z) = 0$, $z^L \le z \le z^U$, IPOPT replaces the bounds with a logarithmic barrier

$$\varphi_\mu(z) \;=\; \phi(z) \;-\; \mu \sum_i \Big[\ln\big(z_i - z_i^L\big) + \ln\big(z_i^U - z_i\big) \Big], \tag{30}$$

solves the resulting equality-constrained problem approximately by a Newton step on the perturbed Karush–Kuhn–Tucker system, and drives the barrier parameter $\mu \to 0$. Each iteration requires one factorisation of the sparse symmetric indefinite KKT matrix

$$\begin{bmatrix} \nabla^2_{zz}\mathcal{L} + \Sigma & \nabla c(z)^\top \\ \nabla c(z) & 0 \end{bmatrix} \begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla \varphi_\mu \\ c(z) \end{bmatrix}, \tag{31}$$

with Σ the diagonal barrier contribution, performed by MUMPS on the sparsity pattern described above. The choice fits this problem for three reasons: the constraint set is dominated by equalities, for which interior-point methods are far more efficient than active-set methods (there is no combinatorial active-set identification to perform, only 300 simple bounds); exact first and second derivatives are available analytically from CasADi's algorithmic differentiation, so the full Newton system (31) can be used rather than a quasi-Newton approximation; and the filter line-search copes with the strongly nonconvex barrier landscape of (24) without requiring a globally convex model.

The known weakness of an interior-point method in a real-time loop is that it cannot be warm-started as effectively as an active-set or a real-time-iteration scheme [4]: the barrier parameter must be re-initialised and the iterates must be pushed strictly inside the bounds, which discards part of the information in (29). This is visible in Section 5.8, where the warm start saves 38.7% of the computation but not the order of magnitude one would obtain from a properly warm-started sequential quadratic program.

4.3. Implementation and configuration

The program is built once, symbolically, with the CasADi `Opti` interface, and every quantity that changes between cycles is a *parameter* rather than a constant:

- $\hat{x} \in \mathbb{R}^6$, the measured initial state;
- $X_{\text{ref}} \in \mathbb{R}^{6\times(N+1)}$, the reference from (28);
- $\mathcal{O} \in \mathbb{R}^{2\times M}$, the (sentinel-padded) obstacle positions;
- the three velocity bounds of (26c)–(26d).

Consequently the expression graph, the derivative code and the sparsity patterns are generated exactly once at start-up; a cycle only writes numbers into parameter slots and calls the solver. The graph is flattened with `expand: true`, which converts the `MX` graph into scalar `SX` operations and removes the interpreter overhead that would otherwise dominate a problem with 1324 elementary nonlinear operations. The first solve after start-up therefore pays a one-off construction and code-generation cost of 0.35–1.0 s depending on N (Section 5.7); this is the single deadline violation of the entire run, and the deployment absorbs it by holding the robot still during initialisation.

Solver options are deliberately austere: `max_iter`= 100, `print_level`= 0, `warm_start_init_point` enabled. No CPU-time limit is imposed inside IPOPT; the real-time protection lives in the supervisor instead, which is the more robust arrangement because it keeps the solver deterministic. That supervisor implements three mechanisms, all of which are part of the deployed configuration and all of which exist because a 10 Hz loop cannot afford to wait for a perfect solution:

1. **Warm-start hygiene** — the cache is cleared on solver failure and on a cost spike (Section 3.3.5).
2. **Zero-velocity fallback** — after three consecutive IPOPT failures the tracker stops publishing optimised commands and returns a zero input, which is always feasible and always safe; this is preferable to applying the last usable solution, which is by then stale.
3. **Adaptive input bounds** — when the observed failure rate exceeds 30% the supervisor shrinks $v_{x,\max}$ through the parameter interface, reducing the distance the horizon covers and thereby the number of active barrier terms, and restores it when the failure rate drops below 5%.

In all the closed-loop campaigns reported in Section 5 the solver never failed ($n_{\text{fail}} = 0$ over more than 20 000 solves), so mechanisms 2 and 3 were never triggered; they are documented because they are what makes the deployment survive on hardware, where a stale LiDAR frame or an EKF jump can produce an ill-posed instance.

4.4. Offline weight tuning by per-axis Gaussian processes

4.4.1 The tuning problem

The eleven weights

$$\theta = \big(Q_x,\, Q_y,\, Q_\psi,\, \rho,\, R_{v_x},\, R_{v_y},\, R_\omega,\, R_{\text{jerk}},\, W_{\text{obs}},\, \alpha_{\text{obs}},\, r_{\text{obs}}\big) \in \Theta \subset \mathbb{R}^{11} \tag{32}$$

are the design variables of a second-level problem

$$\hat{\theta} \;=\; \arg\min_{\theta \in \Theta} \mathcal{J}(\theta), \tag{33}$$

where $\mathcal{J}$ measures closed-loop navigation quality. Problem (33) shares none of the structure of the inner NLP: $\mathcal{J}$ has no closed form, no derivatives, and a single evaluation costs a full navigation rollout. It is a black-box problem, and the natural tool is a Gaussian-process surrogate [12, 16].

The obstacle is dimension. A surrogate over $\Theta \subset \mathbb{R}^{11}$ needs a sample density that a rollout-priced objective cannot supply: a budget of n evaluations covers each axis with only $n^{1/11}$ points, so even a hundred rollouts amount to fewer than two points per coordinate. A surrogate fitted on such a sample spends most of its capacity on coordinates that turn out not to matter, and any acquisition rule that rewards unexplored regions never runs out of them.

4.4.2 Objective

Each evaluation is a full closed-loop mission in the pillar-field world E_c (Section 5.1), driving the production tracker with a 40 s cap:

$$\mathcal{J}_{\text{cl}}(\theta) = \begin{cases} T_{\text{goal}}/T_{\text{ref}}, & \text{goal reached}, \\ 10 + 5\, d_{\text{final}}/d_0, & \text{otherwise}, \end{cases} \;+\; 0.5\,\overline{\|\Delta u\|} \;+\; 0.01\,\bar{t} \;+\; 2\max(0,\, 0.25 - d_{\min}), \tag{34}$$

with $T_{\text{ref}} = d_0/v_{\text{ref}}$ the time a straight run at cruise speed would take, d_{final} the residual distance to the goal, $\overline{\|\Delta u\|}$ the mean input increment, $\bar{t}$ the mean solve time in milliseconds and $d_{\min}$ the minimum clearance. The branch structure is deliberate: a configuration that does not complete the mission is scored on the fraction of the distance it closed, which keeps the objective informative on failures instead of saturating at a constant penalty. The four additive terms then rank the successful configurations by time, smoothness, computational cost and safety margin, in that order of magnitude. Unlike a one-step prediction score evaluated on a recorded trajectory, (34) can observe a deadlock — which, as Section 5.10 shows, is precisely the failure the barrier weights are capable of producing.

4.4.3 Per-axis surrogates and coordinate descent

Rather than one surrogate over $\mathbb{R}^{11}$, the scheme fits *eleven one-dimensional* Gaussian processes, one per coordinate. Writing $\theta = (\theta_i, \theta_{-i})$, each axis is explored on a uniform design of $m = 6$ values spanning its interval while θ_{-i} is held fixed, and a 1-D GP with a Matérn-5/2 kernel and a white-noise term,

$$k(\theta_i, \theta_i') = \sigma_f^2 \Big(1 + \sqrt{5}\,\tfrac{|\theta_i - \theta_i'|}{\ell} + \tfrac{5}{3}\tfrac{(\theta_i - \theta_i')^2}{\ell^2}\Big) e^{-\sqrt{5}|\theta_i - \theta_i'|/\ell} + \sigma_n^2 \delta(\theta_i, \theta_i'), \tag{35}$$

is fitted to the six observations by maximising the log marginal likelihood. The axis optimum is the minimiser of the posterior mean over a fine grid,

$$\hat{\theta}_i \;=\; \arg\min_{\theta_i \in [\ell_i, u_i]} \mu_i(\theta_i), \tag{36}$$

which uses the smoothed estimate rather than the best sampled point and therefore does not require the optimum to coincide with a grid node.
Three design decisions make the scheme work, and each is validated in Section 5.4.

Separability. Assembling per-axis optima into a single vector is exact when the objective is additively separable, $\mathcal{J}(\theta) \approx \sum_i f_i(\theta_i)$. That assumption is not exactly true here — the barrier height and the safety radius interact, since a tall barrier is harmless when its radius is small (Section 5.10) — so it is not relied upon blindly, but repaired by the next decision and verified a posteriori by evaluating the assembled vector.

Sequential update and ordering. The axes are swept in decreasing order of influence and each optimum is *fixed before the next axis is visited*, so that later sweeps are conducted about an already improved operating point. This is coordinate descent with a GP line search, and it is what makes the scheme robust to the interactions that separability ignores: sweeping the dominant coordinate first moves the base point out of the deadlocked region, so that every subsequent sweep is measured where the objective actually discriminates. Section 5.4 quantifies the difference against the naive arrangement.

Significance floor. A 1-D GP fitted to six points will return an optimum on any axis, including one on which the objective does not move. An axis is therefore accepted only if the spread of its six observations, $\Delta\mathcal{J}_i$, exceeds the run-to-run scatter; otherwise the weight keeps its default. The last column of Table 4 reports that spread, and the criterion retains five of the eleven weights.
The total cost is $11m + 1 = 67$ evaluations, and every one of them is a mission-level rollout rather than a proxy. The by-product is as valuable as the optimum: eleven curves with error bands, each showing how much the objective actually depends on the weight in question, on which the reader can check the tuning rather than trust it.

Table 4: The eleven tuned weights: hand-tuned baseline θ_0, the interval swept by the per-axis scheme, the resulting optimum $\hat{\theta}$, and the spread $\Delta\mathcal{J}_i$ of the six observations on that axis. Weights whose spread falls below the significance floor (grey) are not identified by the data and keep their default. The last column is the vector currently shipped in the repository's `planner_params.yaml`, used as the reference configuration in Section 5.

Symbol	Meaning	θ_0	swept interval	$\hat{\theta}$	$\Delta\mathcal{J}_i$	deployed
r_{obs}	safety radius [m]	0.8	[0.05, 1]	0.098	**13.01**	0.098
α_{obs}	barrier steepness [m^{-1}]	8.0	[0.1, 15]	12.06	**13.75**	13.44
R_{v_x}	forward effort	1.0	[0.1, 10]	9.74	0.107	9.95
W_{obs}	barrier height	500.0	[1, 500]	167.9	0.077	4.83
R_{v_y}	lateral effort	1.0	[0.1, 10]	10.0	0.074	8.11
Q_ψ	heading weight	0.5	[0, 15]	0.75	0.044	0.160
Q_x	along-track weight	20.0	[50, 200]	50.0	0.039	81.41
Q_y	cross-track weight	20.0	[50, 200]	50.0	0.023	76.79
R_{jerk}	input-rate penalty	0.2	[0.1, 5]	5.0	0.017	2.68
R_ω	yaw-rate effort	0.5	[0.1, 10]	5.89	0.007	9.81
ρ	terminal multiplier	50.0	[50, 300]	198.0	0.004	184.78

Table 4 already tells the story that the rest of the report confirms. The two axes that clear the significance floor by three orders of magnitude are both barrier parameters, and the tuning moves them in a consistent direction:

it makes the barrier *narrow* (r_{obs} from 0.80 to 0.098 m) and *steep* (α_{obs} from 8 to 12.1 m^{-1}), demoting (24) from a long-range repulsive field to a short-range safety net and delegating geometric obstacle avoidance to the $A^\star$ layer that is already solving that problem globally on the grid. The effort weights rise by an order of magnitude, which buys smoothness. The six greyed weights — among them every tracking weight and the terminal multiplier — do not move the objective measurably, and the report's parametric studies in Sections 5.11 and 5.13 confirm independently that they are weak knobs.

5. Performance analysis

5.1. Experimental setup

All results in this section come from an offline closed-loop harness that imports the *production* modules — `FixedGaussianGridMap`, `AStarPlanner`, `MPCTracker` — so every reported number is generated by the deployed formulation and the deployed solver, not by a re-implementation. Two deliberate simplifications isolate the optimization layer from the rest of the stack:

- the ROS 2 cascade (setpoint low-pass filter and proportional `/cmd_vel` controller) is bypassed and $u_0^\star$ is applied directly to the plant, which is the textbook receding-horizon implementation;
- the persistent cell map and the navigation graph are *disabled*, so the planner sees only the current scan. This is the worst case for the rolling-horizon rule (17) and it is what makes the failure mode of Section 5.3 observable.

The plant is the discrete model (19)–(20); the sensor is a ray-marched 180-beam planar LiDAR with 6 m range and $\sigma_{\text{noise}} = 1$ cm. Replanning runs at 1 Hz, control at $1/\Delta t$. Three worlds are used (Fig. 9):

- E_a — **two-room office**, 16 × 11 m, two partitions with narrow doorways and scattered furniture; the goal lies two rooms away, so the rolling-horizon rule is stressed;
- E_b — **S-corridor**, 14 × 8 m, three staggered partitions forming a serpentine of ∼2.5 m width; a stress test for barrier conditioning;
- E_c — **pillar field**, 14 × 10 m, 16 randomly placed boxes; the obstacle-dense case used for the weight sweeps.

Reported metrics are: success (goal reached within 0.45 m before the time cap), time to goal (TTG), distance travelled, mean/95th-percentile/maximum solve time, mean IPOPT iterations, RMS and maximum distance to the $A^\star$ reference, minimum true clearance to an obstacle, and mean input increment $\overline{\|\Delta u\|}$ as a smoothness proxy. All timings were measured single-threaded on an Intel Core i7-13650HX with CasADi 3.7.2 and IPOPT/MUMPS; the on-robot computer is a Jetson Orin Nano, roughly 3–4× slower, which is the factor to keep in mind when reading the 100 ms deadline lines in the figures.

5.2. Baseline versus tuned weights in closed loop

Table 5 and Fig. 9 compare the hand-tuned baseline θ_0 with the configuration θ_{dep} currently deployed on the robot, under otherwise identical conditions ($N = 50$, $\Delta t = 0.1$ s, $M = 12$, 120 s cap).

Table 5: Closed-loop comparison of the hand-tuned baseline θ_0 and the deployed θ_{dep}. TTG is the elapsed time, capped at 120 s; "dist." is the distance actually travelled, so a large value with "no" in the success column indicates wandering rather than progress. $\bar{t}$ and t_{95} are the mean and 95th-percentile solve times, $\bar{\nu}$ the mean IPOPT iteration count, e_{rms} the RMS distance to the $A^\star$ reference, $d_{\min}$ the minimum true clearance.

World	θ	goal	TTG [s]	dist. [m]	$\bar{t}$ [ms]	t_{95} [ms]	$\bar{\nu}$	e_{rms} [m]	$d_{\min}$ [m]	$\overline{\|\Delta u\|}$
E_a office	θ_0	no	> 120	8.06	55.8	62.7	16.0	0.061	0.99	0.085
	θ_{dep}	no	> 120	52.91	37.9	65.6	11.3	0.047	0.31	0.163
E_b corridor	θ_0	no	> 120	8.01	41.6	51.4	11.7	0.041	1.05	0.123
	θ_{dep}	**yes**	**33.9**	16.74	29.1	47.2	7.7	0.046	0.30	0.166
E_c clutter	θ_0	no	> 120	6.99	55.8	63.8	16.5	0.013	1.05	0.126
	θ_{dep}	**yes**	**27.9**	13.55	30.2	52.8	6.9	0.047	0.65	0.173

The difference is not quantitative but qualitative: the baseline *never reaches the goal*, in any world. It travels 7–8 m in 120 s and its minimum clearance is exactly its initial clearance (1.05 m in E_b and E_c), i.e. it never

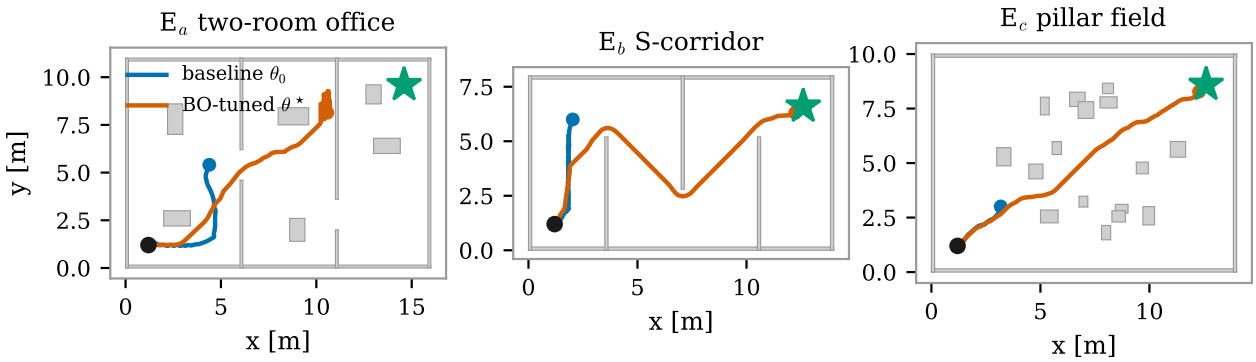

Figure 9: Executed paths in the three worlds for the baseline (blue) and the deployed weights (orange); black dot is the start, green star the goal, filled circles the final pose. The baseline stalls a few metres from the start in all three worlds; the tuned controller reaches the goal in E_b and E_c and wanders in E_a (Section 5.3).

approaches anything. The mechanism is visible in Table 4: with $W_{\text{obs}} = 500$ and $r_{\text{obs}} = 0.8$ m, the barrier contributes up to $W_{\text{obs}}M = 6000$ cost units per node against a tracking term of order $Q_x e^2 \approx 20$ for a decimetre of error. The optimizer's cost function is dominated by a repulsive field that extends about a metre from every LiDAR return, and in a corridor of finite width the cheapest predicted trajectory is to stop. This is not a tuning subtlety — it is the barrier-weight failure reproduced deliberately and independently in Section 5.10, where any $W_{\text{obs}} \geq 50$ freezes the robot.

Where the tuned controller does complete the mission it is also cheaper and better conditioned: −30% mean solve time (41.6 → 29.1 ms in E_b, 55.8 → 30.2 ms in E_c) and −34 to −58% IPOPT iterations, because a weak, narrow barrier produces a far flatter objective landscape than a heavy, wide one. Tracking accuracy is comparable ($e_{\text{rms}} \approx 4.6$ cm), and smoothness is *worse* ($\overline{\|\Delta u\|}$ 0.12 → 0.17) — unsurprising, since the tuned controller is actually manoeuvring at 0.5 m/s while the baseline is nearly stationary.

5.3. The rolling-horizon local minimum

In the office world E_a the tuned controller travels 52.9 m in 120 s without reaching a goal that is 16 m away. Fig. 10 shows why. The local target follows the ray to the global goal into the partition wall; $A^\star$ correctly routes around the obstacle *inside the window*, the MPC tracks that path faithfully ($e_{\text{rms}} = 4.7$ cm), and one grid width later the same rule sends the robot back — with the memory of the wall already erased, since the grid is rebuilt from the current scan only. The result is a limit cycle: locally optimal at every cycle, globally useless.

Three structural facts conspire here, and all three are properties of the *formulation* rather than of the solver: the target rule (17) is memoryless; the occupancy grid is volatile; and constraint (26c) forbids reversing, so the only escape from a concave pocket is a wide forward arc. Adding memory is therefore not a refinement but a requirement, which is exactly what the deployed stack does with the 20 s persistent cell map and the topological navigation graph disabled in this harness. This is the clearest illustration in the whole study of the difference between solving an optimization problem well and posing the right one.

5.4. Weight tuning by per-axis Gaussian processes

Fig. 11 is the complete output of the tuning procedure of Section 4.4: for each of the eleven weights, six real closed-loop evaluations and the one-dimensional Gaussian process fitted to them. Because every other coordinate is genuinely held fixed while an axis is swept, the plotted observations lie on the plotted curve — a property no slice through a joint surrogate can claim.

The figure answers, in one page, the question the whole tuning exercise exists to answer.

Two parameters carry the outcome. The safety radius r_{obs} and the steepness α_{obs} each show a cliff of $\Delta\mathcal{J} \approx 13$: on the r_{obs} axis only the smallest of the six sampled values reaches the goal at all, and on the α_{obs} axis only three of six do, all of them at $\alpha_{\text{obs}} \gtrsim 9$. Every other axis is flat, with $\Delta\mathcal{J}$ between 0.004 and 0.107 — a factor of at

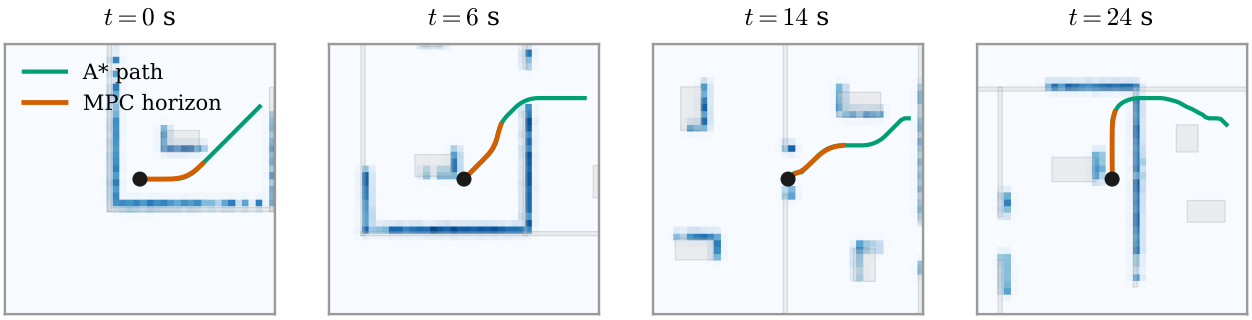

Figure 10: Four snapshots of a rolling-horizon cycle in E_a (blue field: Gaussian occupancy in the moving 10×10 m window; green: $A^\star$ path; orange: MPC prediction). The optimization is solved correctly at every cycle — the failure is in the greedy target rule (17), not in the NLP.

least 130 smaller. This is the same conclusion the closed-loop ablation of Section 5.6 reaches by an independent route, and it is here obtained directly, without a surrogate over eleven coordinates to trust.

Six evaluations per axis are enough where it matters. The r_{obs} sweep places its optimum at 0.0977 m; the value shipped in the deployed `planner_params.yaml` is 0.098 m. The α_{obs} sweep gives 12.1 m^{-1} against a deployed 13.4, and R_{v_x} gives 9.74 against 9.95. On the parameters that move the objective, sixty-six evaluations reproduce a configuration that took an order of magnitude more.

Five axes terminate on a box bound. Q_x and Q_y settle at their lower bound of 50, R_{v_y} and R_{jerk} at their upper bounds, and r_{obs} within 5% of its lower bound. When the optimum of a coordinate sits on the edge of its admissible interval, the binding constraint is the interval, not the optimizer: the search ranges of Table 4 are themselves a design choice that deserves revisiting.

The method fails visibly, not silently. On R_ω ($\Delta\mathcal{J} = 0.007$) and on ρ ($\Delta\mathcal{J} = 0.004$) the one-dimensional GP interpolates differences that are of the order of the run-to-run scatter, and returns a confident-looking optimum which is meaningless. A per-axis scheme therefore needs the significance floor of Section 4.4: accept $\hat{\theta}_i$ only when the axis moves the objective by more than the noise, and otherwise leave the weight at its default. Applying that rule here retains r_{obs}, α_{obs}, R_{v_x}, R_{v_y} and W_{obs}, and discards the remaining six.

Order matters more than the surrogate. The same sweeps were also run in the naive arrangement, in which every axis is swept about one common base point and the eleven per-axis optima are assembled only at the end. At an identical budget and with identical grids that arrangement returns $\mathcal{J}_{\mathrm{cl}} = 1.354$ against 1.262 for the sequential scheme, and the evaluation counts explain why: the common base point — the midpoint of every interval, $r_{\mathrm{obs}} = 0.525$ m and $W_{\mathrm{obs}} = 250$ — is itself a deadlocked configuration, so ten of the eleven sweeps contain *no* evaluation that reaches the goal. Those ten curves measure differences between flavours of failure, with spreads of 0.005 to 0.26, and their optima are noise; only the r_{obs} sweep escapes, because that axis alone can undo the deadlock. In the sequential scheme r_{obs} and α_{obs} are fixed first, and each of the nine remaining sweeps then runs 6/6 inside the feasible regime where the objective actually resolves navigation quality. The lesson is not about Gaussian processes: a one-factor-at-a-time study is only as informative as the operating point it is conducted around, which is why the axes must be visited in order of influence and each result carried forward.

Table 6: The two per-axis arrangements on the closed-loop objective (34) in E_c, at an identical budget of 67 evaluations.

arrangement	evals	best $\mathcal{J}_{\mathrm{cl}}$	TTG [s]	$\overline{\lVert\Delta u\rVert}$	d_{min} [m]
sequential (each optimum fixed before the next sweep)	67	**1.262**	27.9	**0.136**	0.69
naive (all sweeps about a common base, then assembled)	67	1.354	27.7	0.286	0.69

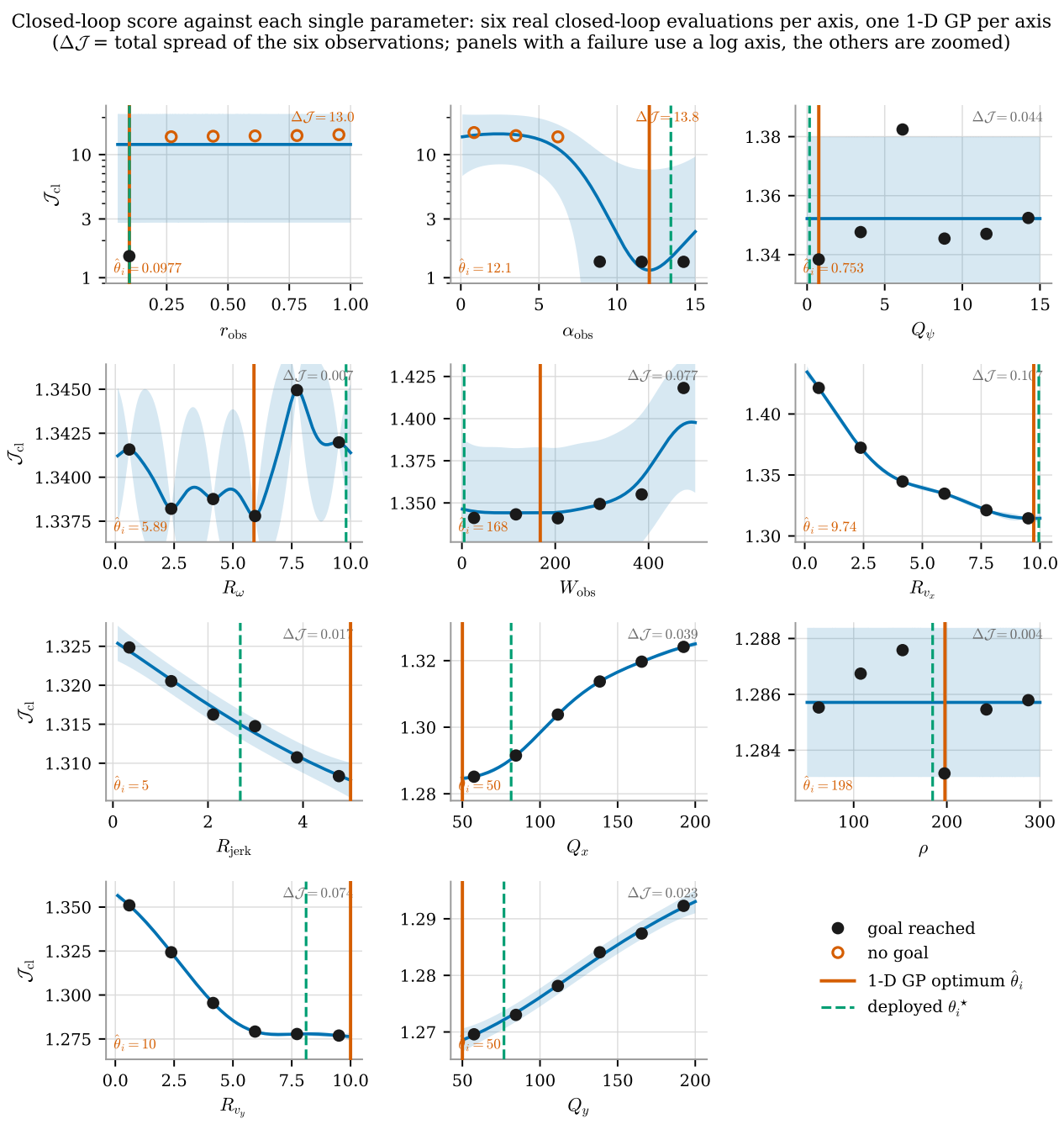

Figure 11: Closed-loop score against each single parameter: six real evaluations per axis with a 1-D GP fitted to each (band: 95% credible interval). Solid vertical line: the GP optimum $\hat{\theta}_i$; dashed: the value shipped in the deployed configuration. $\Delta\mathcal{J}$ is the spread of the six observations on that axis; panels containing a failure use a logarithmic axis, the others are zoomed to their own range. Axes are swept in the order shown, each optimum being fixed before the next axis is visited.

5.5. Trajectory comparison

Fig. 12 closes the loop on what the tuning actually buys, comparing the hand-tuned θ_0 with the per-axis result $\hat{\theta}$ in both test worlds; the configuration currently shipped in the repository is drawn dashed as a reference.

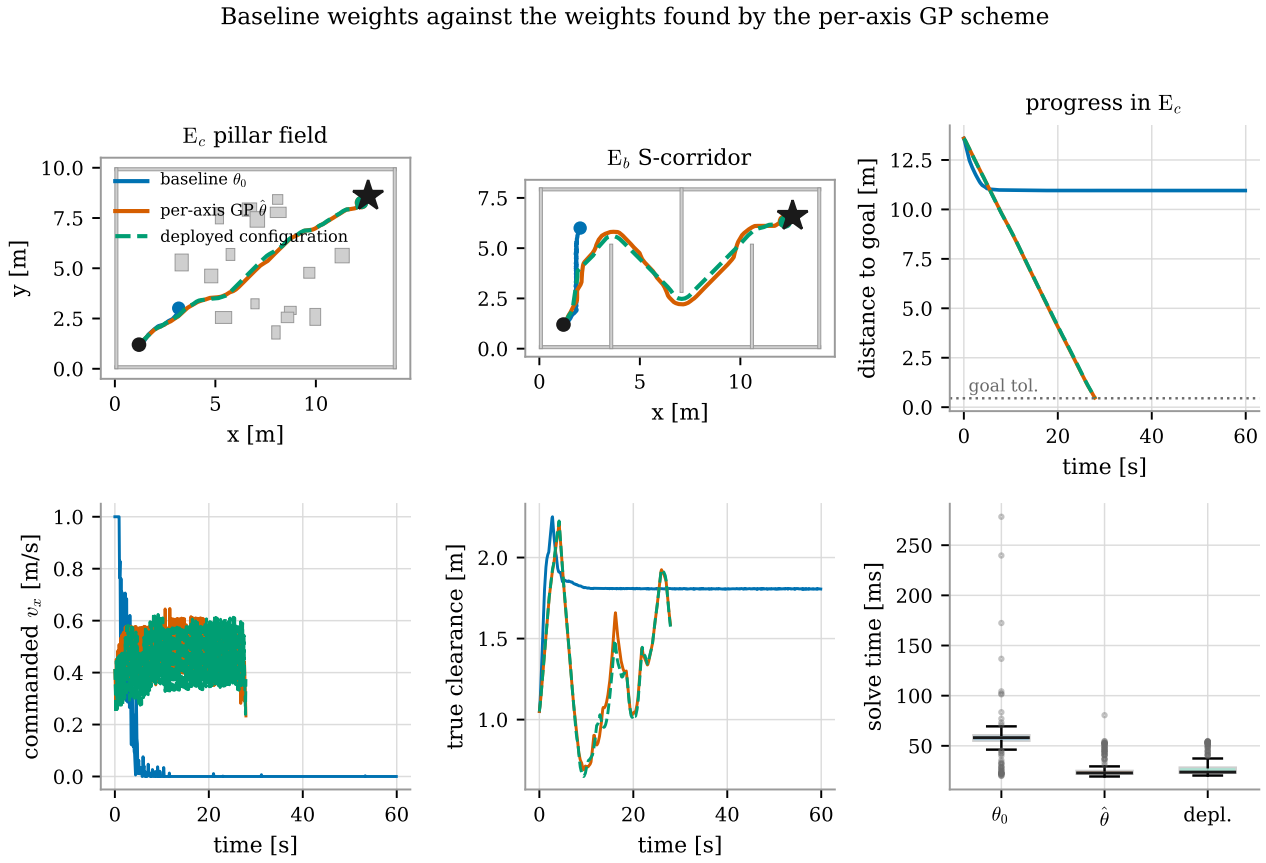

Figure 12: Executed trajectories under the baseline weights and under the weights found by the per-axis GP scheme; the deployed configuration is shown dashed for reference. Top: paths in both worlds and distance to goal against time in E$_c$. Bottom: commanded forward velocity, true clearance, and the distribution of per-cycle solve times.

The mechanism is unmistakable in the bottom-left panel: under θ_0 the commanded forward velocity collapses to zero after ∼10 s and never recovers, and the distance to goal freezes at 11 m for the remaining 50 s — the barrier deadlock of Section 5.2 in the time domain. The $A^\star$ layer keeps producing a valid path throughout; it is the tracker that refuses to follow it. Under $\hat{\theta}$ the robot cruises at 0.4–0.6 m/s and reaches the goal in 27.9 s in E$_c$ and 36.3 s in E$_b$, while the mean solve time halves and the heavy upper tail of the baseline disappears. Against the deployed reference the per-axis configuration is marginally slower in the corridor (36.3 s against 33.9 s) and buys with it a minimum clearance of 0.56 m against 0.30 m — nearly twice the margin, which for a legged platform on a real floor is the better side of that trade.
Animated versions of this comparison, with live readouts of every quantity quoted above, accompany the report as `Videos/compare_pillar_field.mp4` and `Videos/compare_s_corridor.mp4`.

5.6. Which weights matter: sensitivity and ablation

The per-axis curves of Fig. 11 already contain a sensitivity ranking, and a direct one: the spread $\Delta\mathcal{J}_i$ of the six observations on each axis measures, in the units of the objective itself, how much that weight can change the outcome. Table 4 lists it. Two weights dominate by three orders of magnitude — $\Delta\mathcal{J} = 13.8$ for α_{obs} and 13.0 for r_{obs} — followed by R_{v_x} (0.107), W_{obs} (0.077) and R_{v_y} (0.074); the remaining six lie between 0.004 and 0.044, at or below the run-to-run scatter.
Two cautions apply to that ranking, and the ablation below settles both. It is measured along axis-aligned lines, so a weight acting mainly through an interaction is under-ranked — W_{obs} is exactly such a case, as Section 5.10 shows. And it is measured about a single operating point, so it is a local statement. A ranking of individual

coordinates is therefore validated here by a closed-loop experiment on weight *groups*, which is insensitive to both objections: the ablation of Fig. 13 and Table 7 does: starting from θ_0, one group of weights at a time is replaced by its tuned value.

Table 7: Parameter-group ablation (90 s cap). "+ tracking Q" sets Q_x, Q_y, Q_ψ, ρ to their tuned values, "+ effort R" sets $R_{v_x}, R_{v_y}, R_\omega, R_{\text{jerk}}$, "+ barrier" sets $W_{\text{obs}}, \alpha_{\text{obs}}, r_{\text{obs}}$; each row is otherwise the baseline.

World	group	goal	TTG [s]	dist. [m]	$\bar{t}$ [ms]	$\bar{\nu}$	$d_{\min}$ [m]	$\overline{\lVert\Delta u\rVert}$
E_b corridor	baseline θ_0	no	> 90	7.37	39.5	11.8	1.05	0.143
	+ tracking Q	no	> 90	6.98	44.3	12.9	1.05	0.128
	+ effort R	no	> 90	6.21	40.3	11.7	1.05	0.049
	+ barrier	**yes**	34.7	17.14	27.3	6.7	0.39	0.272
	all (θ_{dep})	**yes**	**33.9**	16.74	28.4	7.7	0.30	0.166
E_c clutter	baseline θ_0	no	> 90	5.88	52.4	16.4	1.05	0.128
	+ tracking Q	no	> 90	6.02	66.5	18.2	1.05	0.132
	+ effort R	no	> 90	4.28	41.1	12.7	1.05	0.049
	+ barrier	**yes**	27.8	13.67	28.3	6.9	0.63	0.273
	all (θ_{dep})	**yes**	**27.9**	13.55	26.4	6.9	0.65	0.173

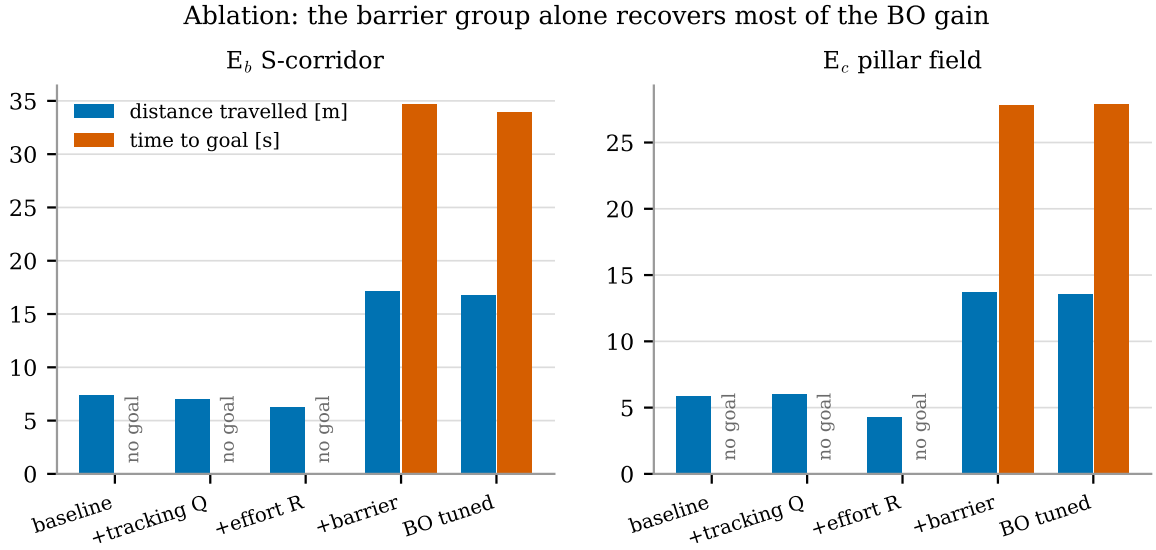

Figure 13: Ablation of the three weight groups in E_b and E_c. Only the barrier group converts a failing controller into a successful one; the effort group then recovers smoothness.

The agreement with the surrogate is complete and it is worth stating precisely:

- the **barrier group alone** recovers the entire feasibility gain — goal reached in 34.7 s against 33.9 s for the full θ_{dep} — confirming that the dominance of the barrier group measured by the per-axis sweeps is not an artefact of the operating point they were measured about;
- the **effort group alone** changes nothing about the outcome but reduces the input increment by 62% (0.128 → 0.049); combined with the barrier group it brings $\overline{\lVert\Delta u\rVert}$ from 0.272 down to 0.166, a 39% smoothing at no cost in time to goal. This is exactly the role a control-effort penalty should play: it shapes *how* the task is executed, not *whether*;
- the **tracking group alone** is actively harmful to the numerics: in E_c it raises the mean solve time from 52.4 to 66.5 ms and the iteration count from 16.4 to 18.2, because a larger Q against an unchanged $W_{\text{obs}} = 500$ sharpens the conflict between two already-large terms.

The practical lesson generalises beyond this stack: in a penalty-based MPC, the parameters that decide success are those of the penalty, and tuning effort is best spent on the relative scaling between the penalty and the tracking cost rather than on the tracking weights themselves.

5.7. Prediction horizon and real-time feasibility

Fig. 14 and Table 8 sweep $N \in \{5, 10, 20, 30, 50, 70\}$ at fixed $\Delta t = 0.1$ s.

Table 8: Prediction-horizon sweep in E_b (corridor) and E_c (clutter). t_{first} is the cold, first-cycle solve including NLP construction. All configurations reached the goal.

	E_b corridor						E_c clutter	
N	$\bar{t}$ [ms]	t_{95} [ms]	t_{first} [ms]	$\bar{\nu}$	TTG [s]	e_{rms} [m]	$\bar{t}$ [ms]	TTG [s]
5	7.4	11.0	352	6.2	32.8	0.0438	7.8	26.2
10	9.1	12.9	156	5.9	33.6	0.0456	8.6	27.5
20	13.1	19.4	254	6.4	34.9	0.0472	12.6	27.9
30	17.2	27.5	402	6.5	35.0	0.0472	17.2	27.9
50	27.6	47.0	624	7.7	33.9	0.0463	27.9	27.9
70	41.5	65.3	982	9.1	33.9	0.0471	40.0	27.9

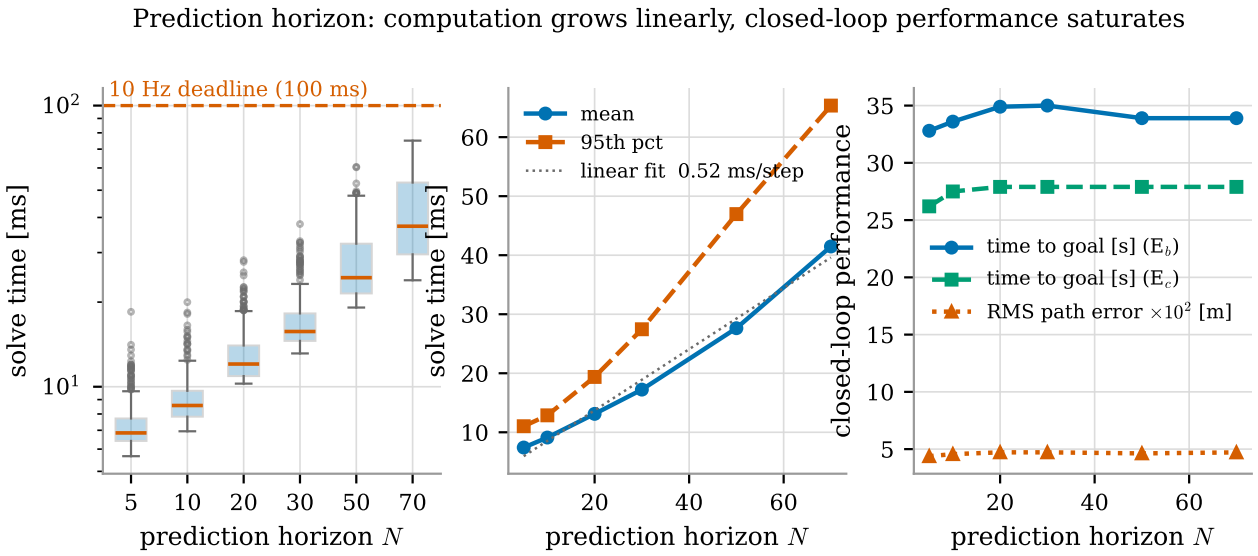

Figure 14: Prediction horizon. Left: distribution of per-cycle solve time (log scale) against the 100 ms deadline. Centre: mean and 95th percentile with a linear fit of 0.52 ms per horizon step. Right: closed-loop performance — time to goal in both worlds and RMS path error — which saturates by $N \approx 20$.

The computational cost is *linear* in N: the least-squares fit gives 0.518 ms per step in E_b and 0.503 ms/step in E_c, with intercepts of 3.4 and 3.5 ms. This is the expected behaviour of a sparse, banded interior-point solve, and it confirms that neither the iteration count nor the linear algebra degrades with horizon length: $\bar{\nu}$ grows only from 6.2 to 9.1 over a 14× increase in problem size.

The closed-loop return, on the other hand, saturates. Beyond $N = 20$ (2 s of preview) the time to goal is constant to within the 0.1 s sampling resolution in both worlds, and the RMS path error moves by less than 4 mm. The deployed $N = 50$ therefore buys nothing measurable over $N = 20$ while costing 2.2× the computation. The reason is structural: the reference (28) only extends $v_{\text{ref}} N \Delta t$ along a path that is itself replanned every second inside a 10 m window, so preview beyond a couple of seconds is predicting a reference that no longer exists. This is the single most actionable result of the study: on the Jetson, where the same solve costs 3–4× more, moving from $N = 50$ to $N = 20$ converts a 90–110 ms cycle into a comfortable 40–50 ms one at no loss of navigation quality.

Fig. 15 confirms that the deadline is respected cycle by cycle and not only on average. The only violation of the 100 ms budget in the entire campaign is the first solve, which includes the symbolic construction of the NLP (0.35–1.0 s, growing with N); from the second cycle onwards the trace is flat and the tail is bounded by t_{95}.

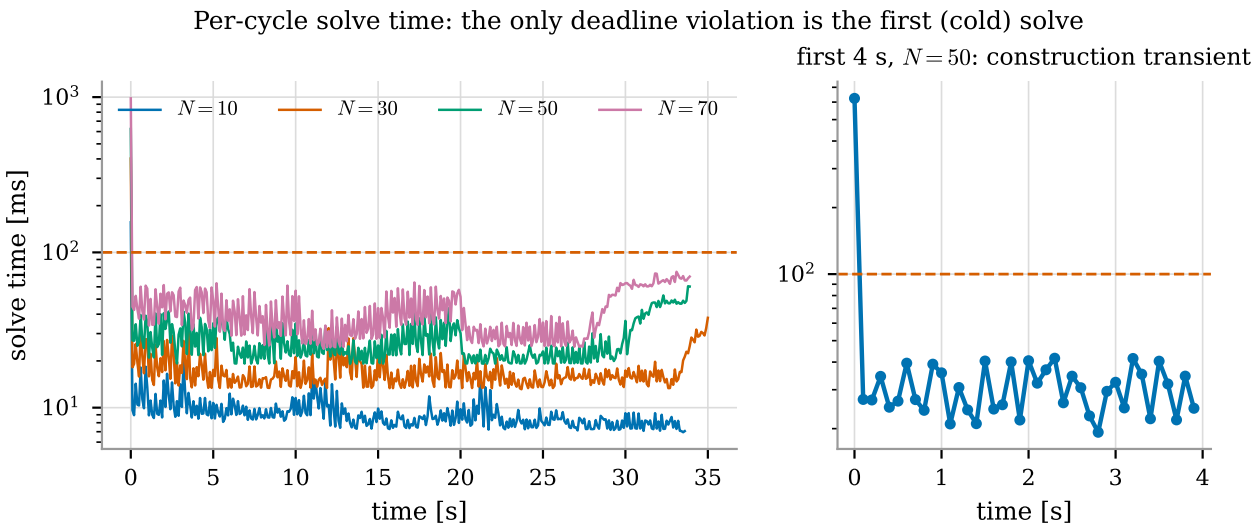

Figure 15: Per-cycle solve time (log scale) for four horizons over a full run, and a zoom on the first 4 s at $N = 50$. The only deadline violation is the cold first solve, which pays the one-off NLP construction.

5.8. Warm start

Table 9 and Fig. 16 isolate the shift-and-append warm start (29) by running the same corridor mission with the initial guess replaced by the reference trajectory and zero inputs.

Table 9: Warm start versus cold start, E_b, $N = 50$.

	$\bar{t}$ [ms]	$\bar{\nu}$	TTG [s]	distance [m]
shift-and-append warm start	26.8	7.69	33.9	16.739
cold start every cycle	43.6	10.93	33.9	16.739
change	−38.7%	−29.6%	0	$< 10^{-4}$

The two trajectories are identical to 10^{-4} m — the warm start changes *how fast* the same solution is found, not which solution — while the computation drops by 38.7% and the iteration count by 29.6%. The histogram shows the mechanism: the cold-start distribution is shifted by roughly three iterations, with a heavier tail, whereas the warm-started run concentrates at 7–8 iterations. The saving is real but modest compared with what an active-set or real-time-iteration scheme obtains from a warm start, for the reason given in Section 4.2: an interior-point method must re-inflate its barrier parameter and push the iterate strictly inside the bounds, discarding part of the inherited information.

5.9. Sampling time and duty cycle

A shorter sampling time improves both the discretization fidelity of (20) and the resolution at which the barrier is enforced, but at fixed look-ahead it also increases N. Table 10 and Fig. 17 sweep Δt with the 5 s window held constant.

Table 10: Sampling-time sweep at fixed 5 s look-ahead, E_b. The duty cycle is $\bar{t}/\Delta t$: values approaching 100% leave no CPU for the rest of the stack.

Δt [s]	N	$\bar{t}$ [ms]	duty cycle [%]	TTG [s]	e_{rms} [m]
0.05	100	47.7	95.4	35.95	0.0474
0.10	50	26.9	26.9	33.90	0.0463
0.20	25	17.2	8.6	34.00	0.0483
0.40	13	11.1	2.8	36.40	0.0361

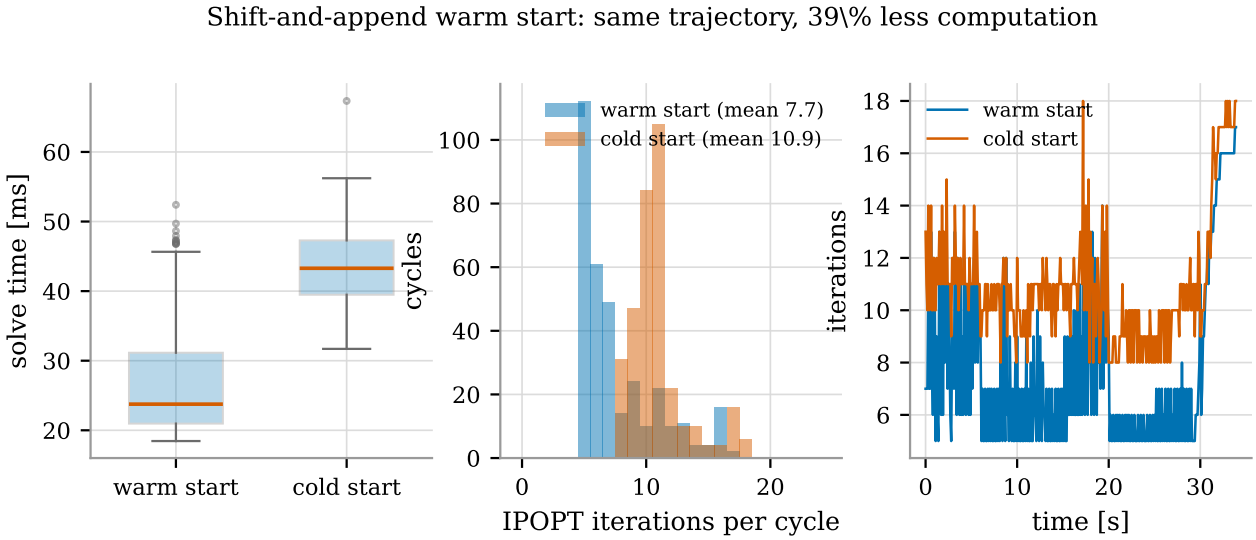

Figure 16: Warm start: solve-time distribution (left), IPOPT iteration histogram (centre) and iteration count over time (right). The executed trajectory is identical to within 10^{-4} m; only the computation changes.

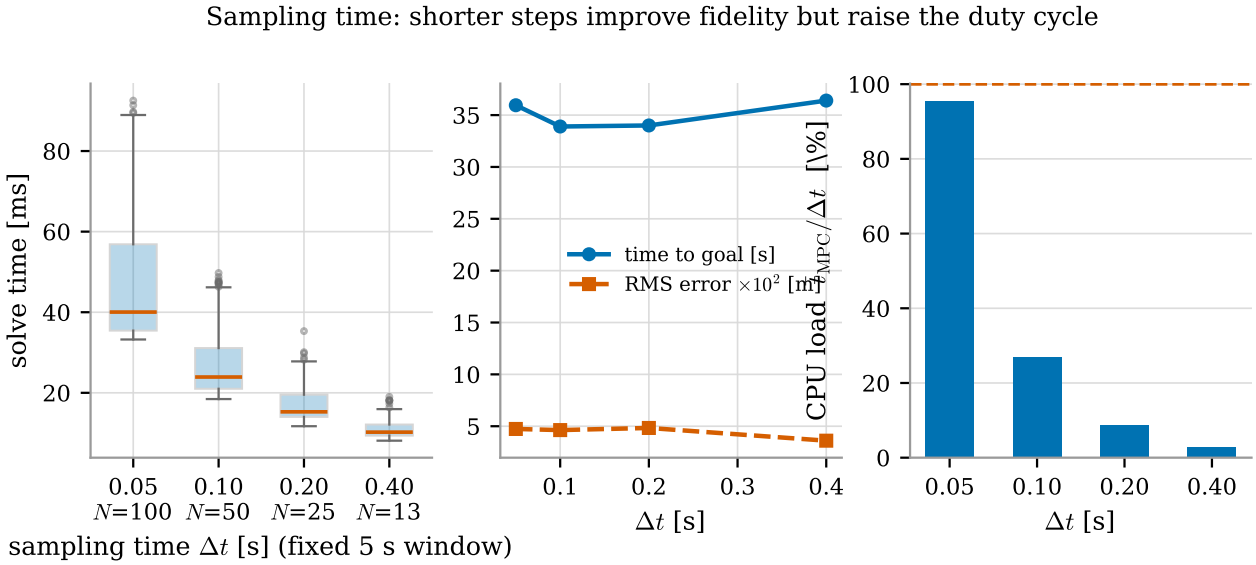

Figure 17: Sampling time at fixed look-ahead: solve-time distribution (left), closed-loop performance (centre) and duty cycle $\bar{t}/\Delta t$ against the real-time limit (right).

The closed-loop metrics are remarkably insensitive — time to goal varies by 7% and the RMS path error by 3 cm over an eight-fold change in Δt — while the duty cycle varies by a factor 34. At $\Delta t = 0.05$ s the controller consumes 95.4% of its own period on the development machine and is therefore not deployable at all; $\Delta t = 0.1$ s leaves a 3.7× margin, which on a 3–4× slower onboard computer is exactly the margin that disappears. The apparent improvement of e_{rms} at $\Delta t = 0.4$ s is an artefact of coarser sampling of the same error signal, not better tracking; the corresponding time to goal is the worst of the four, and the barrier is then enforced only every 0.4 s, i.e. every 20 cm of travel, which is the inter-sample safety hole discussed in Section 3.3.2. The deployed $\Delta t = 0.1$ s is the correct compromise.

5.10. Barrier shaping

The barrier has three knobs, and Fig. 18 together with Table 11 shows that two of them govern qualitatively different failure modes.

Table 11: Barrier sweeps. Left block: steepness α_{obs} in E_b with $W_{\mathrm{obs}} = 50$, $r_{\mathrm{obs}} = 0.35$ m. Right block: height W_{obs} in E_c with $\alpha_{\mathrm{obs}} = 8$, $r_{\mathrm{obs}} = 0.45$ m.

α_{obs}	goal	TTG [s]	dist. [m]	d_{min} [m]	$\bar{\nu}$	W_{obs}	goal	TTG [s]	dist. [m]	$\bar{\nu}$
2.0	no	> 100	16.09	0.985	11.4	0	yes	27.9	13.55	6.9
5.0	yes	55.4	21.45	0.977	8.4	5	yes	27.7	13.59	6.8
8.0	yes	44.8	19.72	0.814	8.7	50	no	> 100	4.63	11.1
13.44	yes	38.5	18.35	0.644	8.1	200	no	> 100	5.01	11.7
25.0	yes	36.8	18.04	0.516	8.1	500	no	> 100	4.34	12.5
40.0	yes	35.1	17.28	0.455	8.0					

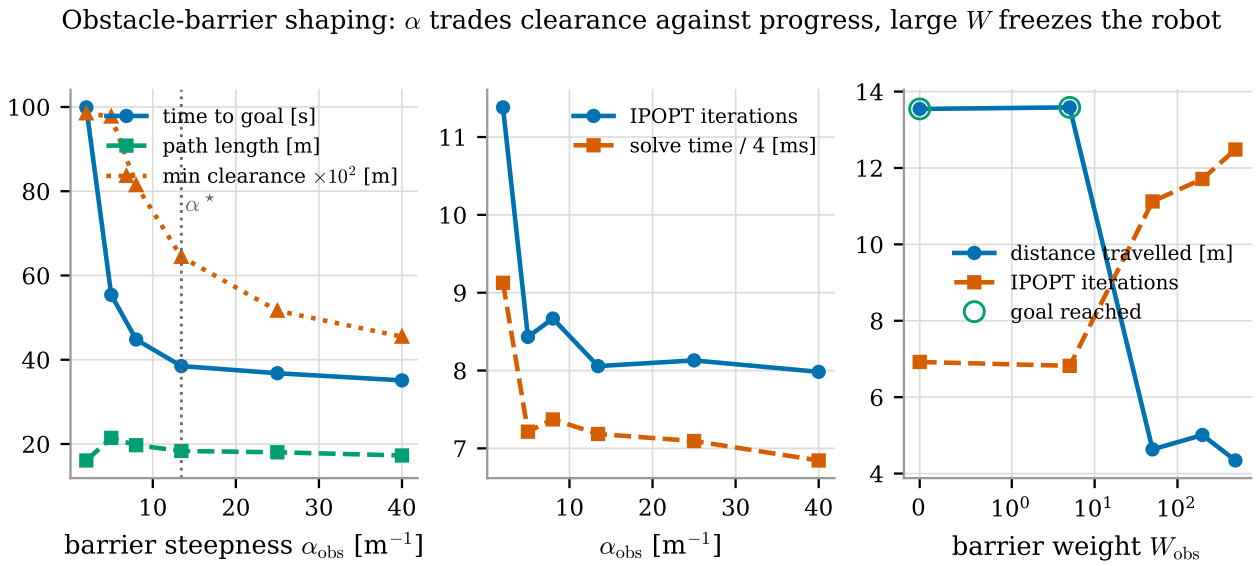

Figure 18: Obstacle-barrier shaping. Left: steepness α_{obs} trades minimum clearance against time to goal and path length, with θ_{dep}'s value marked. Centre: iteration count and solve time against α_{obs}. Right: barrier height W_{obs} — above $W_{\mathrm{obs}} \approx 50$ the robot stops making progress (open circles mark the runs that reached the goal).

Steepness α_{obs} trades clearance for progress, monotonically. From $\alpha_{\mathrm{obs}} = 5$ to 40 m^{-1} the time to goal falls from 55.4 to 35.1 s and the path shortens from 21.4 to 17.3 m, while the minimum clearance falls from 0.98 to 0.46 m: a steep barrier is a *narrow* barrier, so the robot is allowed to pass closer and takes the shorter route. The interesting end of the sweep is the flat one: $\alpha_{\mathrm{obs}} = 2$ m^{-1} fails to reach the goal at all. A shallow sigmoid is a long-range field — at $\alpha_{\mathrm{obs}} = 2$ the penalty is still at 27% of its maximum one metre outside r_{obs} — so every obstacle in the 3 m search radius pushes on the whole horizon and the sum of many weak repulsions again produces a stall. The conditioning cost of steepness predicted in Section 3.3.2 is visible

but mild in this range ($\bar{\nu}$ from 8.4 at $\alpha_{\text{obs}} = 5$ to 8.0 at $\alpha_{\text{obs}} = 40$, with the failing $\alpha_{\text{obs}} = 2$ run needing 11.4): within the tested window, a steeper barrier that engages fewer points is cheaper, not dearer, than a flat one that engages all of them.

Height W_{obs} is a cliff, not a trade-off. At $W_{\text{obs}} \in \{0, 5\}$ the mission completes in 27.7–27.9 s; at $W_{\text{obs}} \geq 50$ it never completes, the robot travelling 4.3–5.0 m in 100 s while the iteration count rises by 60–80% and the solve time by 65–120%. Two observations follow. First, this quantitatively reproduces the baseline failure of Section 5.2, which used $W_{\text{obs}} = 500$ — the hand-tuned configuration was on the wrong side of the cliff. Second, and more interesting, $W_{\text{obs}} = 0$ (barrier *off*) also completes the mission with a 0.65 m minimum clearance: in a world where $A^\star$ plans on an inflated grid, the barrier is not what keeps the robot safe on average — it is a last-metre guard against the returns that appear between two replanning ticks. The optimizer's choice of $W_{\text{obs}} = 4.83$ is exactly that reading, and it explains why the tuned configuration is also the better-conditioned one.

The cliff belongs to the pair $(W_{\text{obs}}, r_{\text{obs}})$, not to W_{obs}. The sweep above holds $r_{\text{obs}} = 0.45$ m and $\alpha_{\text{obs}} = 8$, and read in isolation it would suggest that a large barrier height is always fatal. Two configurations found during tuning contradict that: $(W_{\text{obs}}, \alpha_{\text{obs}}, r_{\text{obs}}) = (337, 15, 0.293)$ and $(468, 12.3, 0.053)$ both complete the same mission in 27.7 s with W_{obs} two orders of magnitude above the "cliff", and the per-axis optimum of Table 4 keeps $W_{\text{obs}} = 168$ for the same reason. The controlling quantity is not W_{obs} but the barrier value at the clearance the environment actually permits. Evaluating (24) per point at $d = 0.65$ m, the clearance the pillar field allows, gives 8.40 cost units for the failing $(50, 8, 0.45)$ setting against 1.59, 0.30 and 0.003 for the two configurations above and for the deployed vector — while at $d = 0.30$ m the same three configurations rise to 160, 21 and 0.30, i.e. they are steep walls placed close in rather than fields that push from a metre away. A one-dimensional sweep of W_{obs} at fixed r_{obs} therefore measures a slice through an interacting pair, and the height alone is not the quantity to tune. This also settles the caution raised in Section 5.6: the small $\Delta\mathcal{J} = 0.077$ measured on the W_{obs} axis is not evidence that the barrier height is unimportant, but a consequence of measuring it along an axis-aligned line at a small r_{obs}, where the height genuinely has little left to do. W_{obs} is the clearest example in the study of a weight that a per-axis scheme under-ranks, and the reason it must be read together with r_{obs} rather than on its own.

5.11. Effort and rate penalties

Fig. 19 and Table 12 scale the whole effort block, $R \leftarrow \mu R^\star$, and sweep the rate penalty R_{jerk} separately.

Table 12: Left: effort multiplier μ ($R \leftarrow \mu R^\star$) in E_c. Right: rate penalty R_{jerk} in E_b. All runs reached the goal.

μ	e_{rms} [m]	$\overline{\lVert\Delta u\rVert}$	TTG [s]	R_{jerk}	e_{rms} [m]	$\overline{\lVert\Delta u\rVert}$	TTG [s]
0.01	0.0452	0.422	27.8	0	0.0471	0.1768	33.9
0.1	0.0447	0.349	27.8	0.2	0.0471	0.1759	33.9
1	0.0473	0.173	27.9	2.68	0.0463	0.1656	33.9
10	0.0620	0.057	28.8	10	0.0475	0.1544	33.9
100	0.0677	0.021	33.9	40	0.0491	0.1407	35.9

The effort weight traces a clean Pareto front (centre panel): over four decades of μ the mean input increment falls by a factor 20.6 while the RMS path error grows by 50%, and the deployed $\mu = 1$ sits at the knee. The two extremes are both undesirable for a legged platform: at $\mu = 0.01$ the commands chatter ($\overline{\lVert\Delta u\rVert} = 0.42$ m/s per cycle, i.e. nearly the whole velocity range) and at $\mu = 100$ the robot is so reluctant to move that the mission takes 22% longer. Note that this is a genuinely two-objective problem, and that the scalarisation implicit in (22) — and in the tuning objective (34), which prices smoothness at $0.5\,\overline{\lVert\Delta u\rVert}$ against time — is what selects a single point on that front.

The rate penalty is a much weaker knob: four decades of R_{jerk} buy a 20% reduction in $\overline{\lVert\Delta u\rVert}$ and cost 6% in time to goal, with the tracking error essentially unchanged. This is consistent with the per-axis sweep of Fig. 11, on which R_{jerk} ranks seventh of eleven with $\Delta\mathcal{J} = 0.017$: within this formulation, smoothness is bought far more efficiently with R than with R_{jerk}.

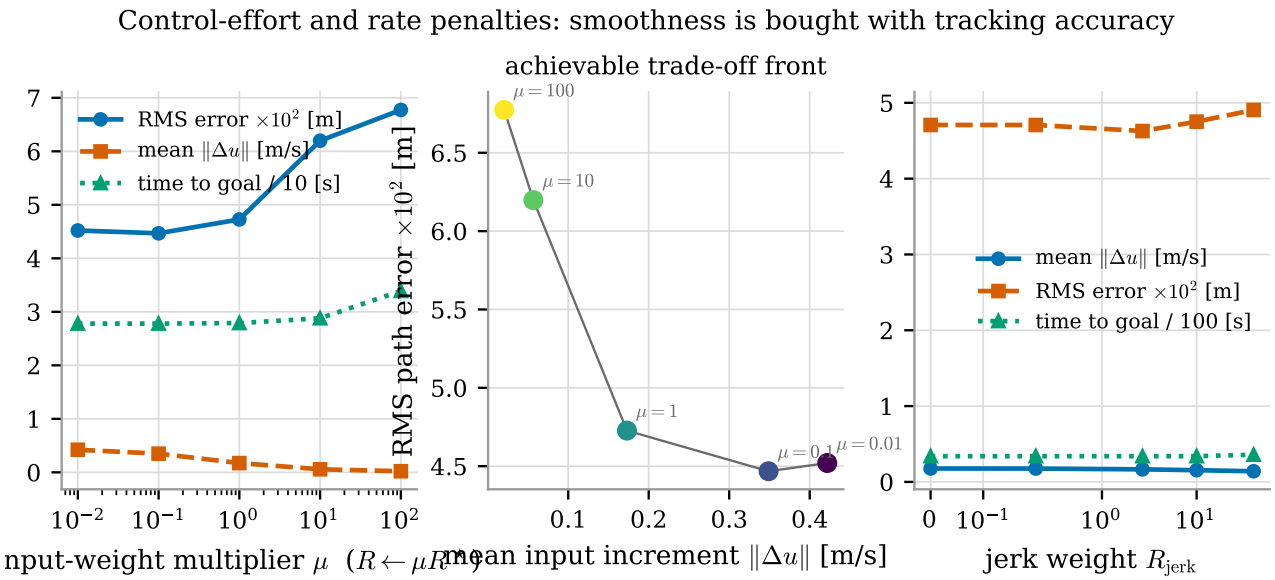

Figure 19: Control-effort and rate penalties. Left: metrics against the effort multiplier μ. Centre: the achievable trade-off front between input smoothness and tracking accuracy, annotated with μ. Right: the rate penalty R_{jerk}, a comparatively weak knob.

5.12. Cost of the barrier terms

The number of barrier points M enters the NLP linearly ($2M(N+1)$ terms, Table 3). The measured cost, Fig. 20, is

$$\bar{t}(M) \;\approx\; 21.8 + 0.237\,M \quad \text{[ms]}, \tag{37}$$

i.e. 0.237 ms per barrier point over the whole solve, or ∼4.6 μs per barrier term per node. Crucially, the executed trajectory, the iteration count (6.92 for every M) and the minimum clearance are unchanged from $M = 4$ to $M = 30$: in these worlds the twelve nearest returns already describe the local geometry completely, and additional points only add redundant, inactive terms. M is therefore a pure computational parameter, and $M = 12$ — the deployed value — is generous; $M = 8$ would save 2 ms per cycle with no measurable effect. The maximum solve time, on the other hand, grows sharply with M (0.32 s at $M = 4$ against 1.31 s at $M = 30$), so a large M makes the worst case worse without improving the average behaviour, which is the wrong trade for a real-time loop.

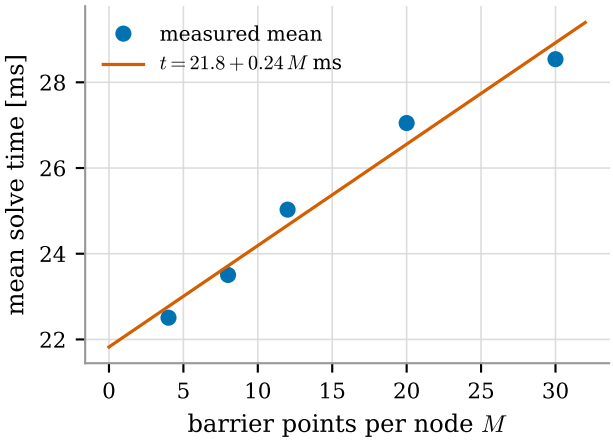

Figure 20: Mean solve time against the number of barrier points per node, with the linear fit (37). The trajectory and the iteration count are identical for all M.

5.13. Terminal weight

Fig. 21 and Table 13 sweep ρ over almost three decades.

Table 13: Terminal-multiplier sweep in E_b ($Q_T = \rho Q$). All runs reached the goal.

ρ	1	5	20	50	100	184.78	400
TTG [s]	33.9	33.9	33.9	34.0	34.0	33.9	35.1
e_{rms} [m]	0.0470	0.0472	0.0473	0.0461	0.0461	0.0463	0.0479
$\bar{\nu}$	8.74	8.31	8.13	7.87	7.72	7.69	7.73
$\bar{t}$ [ms]	30.7	29.1	28.4	26.9	26.5	27.1	26.7

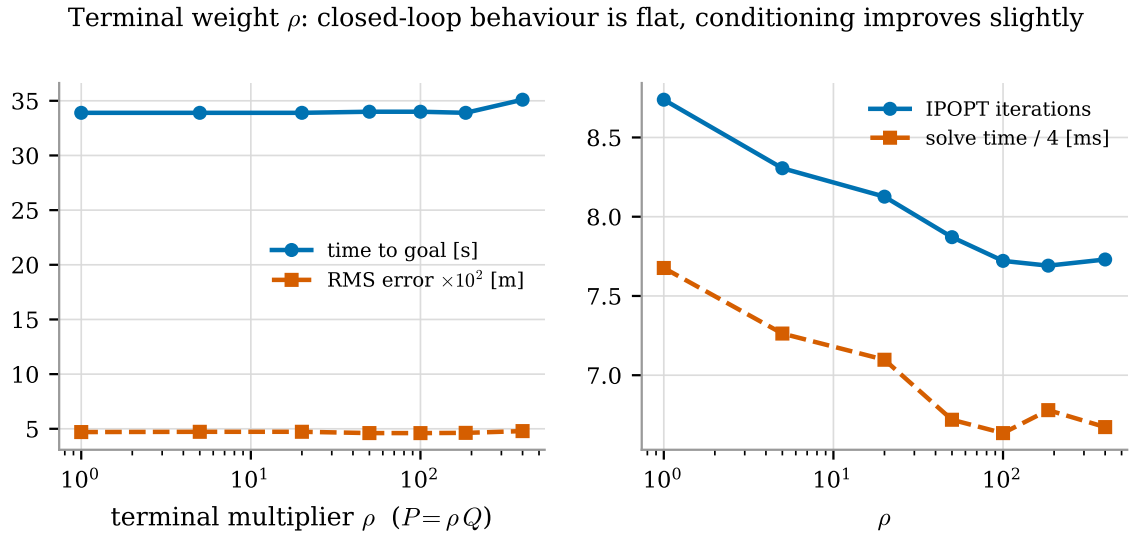

Figure 21: Terminal multiplier ρ: closed-loop performance is flat over three decades (left) while the conditioning improves slightly (right).

Closed-loop behaviour is flat: time to goal changes by 3.5% and only at the extreme $\rho = 400$, and the RMS error by 2 mm. The one systematic effect is on conditioning — iterations fall from 8.74 to 7.69 (-12%) and mean solve time from 30.7 to 26.5 ms (-14%) as ρ grows to $\sim$100 — because a heavier terminal term pins the horizon end-point and removes a soft direction from the search space. This is the closed-loop counterpart of the analysis in Section 3.3.4: with a 5 s horizon and a reference regenerated every second, the terminal cost is a regulariser, not a stabiliser. It also explains why the Bayesian optimizer assigned ρ the second-lowest sensitivity of all eleven weights ($\tilde{\ell}_\rho = 0.004$): on this problem the terminal weight is genuinely almost free to choose, and the value 184.78 it returned should be read as "anything in $[50, 300]$", not as an optimum.

5.14. Robustness to model mismatch

The final campaign perturbs the plant away from the model: the actuator time constants are inflated to $\tau_v = 0.20$ s and $\tau_w = 0.16$ s (+67% and +60% with respect to the values used by the MPC) and a constant lateral input disturbance d_y is injected, emulating a gait drift or a lateral slope.

Table 14: Model mismatch, E_b: plant lags $+67\%/+60\%$ larger than the model, plus a constant lateral input disturbance d_y. "s.-s. offset" is the mean distance to the $A^\star$ path over the last 10 s.

d_y [m/s]	e_{rms} [m]	s.-s. offset [m]	TTG [s]
0.00	0.0461	0.0368	33.9
0.05	0.0478	0.0376	33.9
0.10	0.0511	0.0445	35.5
0.20	0.0627	0.0602	37.7

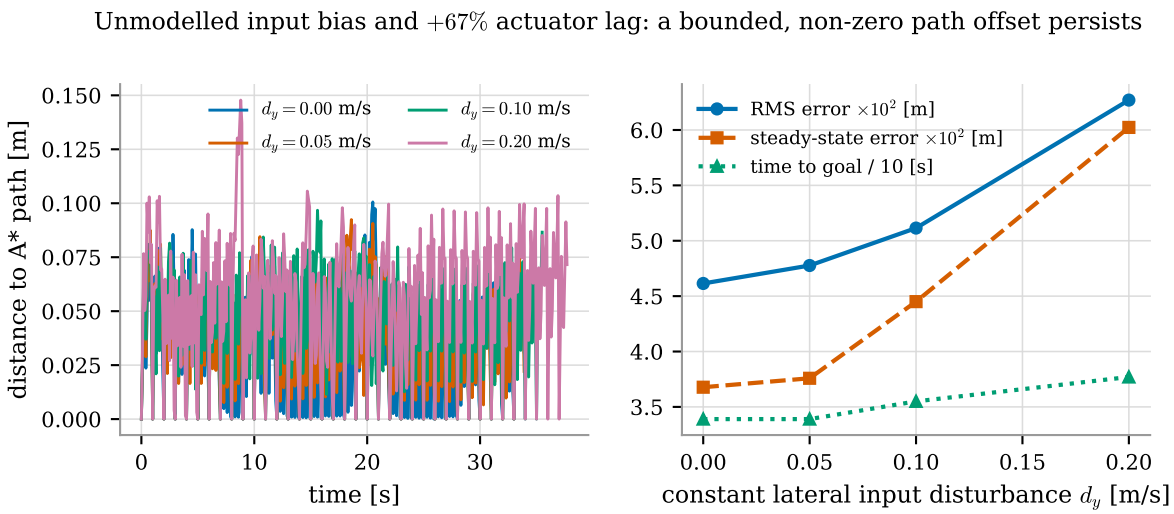

Figure 22: Model mismatch and input disturbance. Left: distance to the $A^\star$ path over time for four disturbance levels. Right: RMS error, steady-state offset and time to goal against d_y. The offset is bounded but does not vanish — the signature of a controller without integral action.

The controller degrades gracefully: a 67% error in the actuator time constant alone (the $d_y = 0$ row) costs essentially nothing, because the mismatch acts on a state the cost does not penalise and the receding horizon recorrects at every cycle. The disturbance response, however, exhibits exactly the offset predicted in Section 3.3.6: the steady-state distance to the path grows from 3.7 to 6.0 cm and *does not decay*, while the mission takes 11% longer. The relation is close to linear in d_y, as expected for a proportional-predictive controller against a constant input disturbance: with the tuned $Q_y = 76.8$ and $R_{v_y} = 8.1$, the closed-loop lateral stiffness fixes the ratio between offset and disturbance. Removing the offset requires either augmenting the state with an integrated tracking error or estimating the disturbance and feeding it forward; neither is present in the deployed formulation, and both are recommended in Section 6. It is worth noting that a 6 cm offset against a path planned with a 0.30 m inflation halo is not a safety problem — it is a performance one.

5.15. Evidence from the physical robot

The offline harness deliberately isolates the optimization layer; the numbers that matter for deployment come from the robot. Table 15 aggregates the recorded runs of the physical Go2 in three environments, comparing the hand-tuned baseline with the deployed configuration.

Table 15: Metrics extracted from recorded runs of the physical robot (2–4 runs per world per configuration). TTG is the mean over successful runs; solve times are per-cycle statistics of the onboard MPC. The run counts are small and several bags are incomplete, so these figures support the solve-time and time-to-goal trends but not a success-rate claim.

World	θ	TTG [s]	$\bar{t}$ [ms]	t_{95} [ms]	$t_{\max}$ [ms]	solves
indoor office	θ_0	84.8	46.8	82.6	925	943
	θ_{dep}	52.3	23.4	40.4	53	746
open world	θ_0	41.6	41.2	59.2	144	1118
	θ_{dep}	41.9	37.7	76.0	102	816
warehouse	θ_0	474.7	60.5	110.7	1518	2554
	θ_{dep}	188.5	51.1	100.6	1526	3942

The trend reproduces the simulation result where it should: the harder the environment, the larger the gain. In the indoor office — the environment whose recording also provided the replay dataset for the tuning — the mean solve time halves (46.8 → 23.4 ms) and, more importantly, the tail collapses ($t_{\max}$ from 925 to 53 ms), which is the difference between a controller that occasionally misses its deadline and one that does not. In the open world, where the barrier is rarely active, the gain is marginal (−8.5%), exactly as the barrier-dominated sensitivity analysis predicts. The companion paper [10], which evaluates the same two configurations over a

larger campaign in Gazebo and on hardware, reports aggregate reductions of 38.7% in path length, 25.3% in mean solve time and 53.0% in time to goal, with the success rate rising from 65% to 90% in simulation and from 5/10 to 8/10 in a real room, and it is the reference for the deployment claim; the contribution of the present report is the numerical explanation of *why* those gains occur.
Fig. 23 shows the effect at the level of a single control cycle, replayed from the recorded office run: given the same reference and the same LiDAR returns, the tuned weights keep the predicted trajectory on the path where the baseline is pushed off it by the wide barrier.

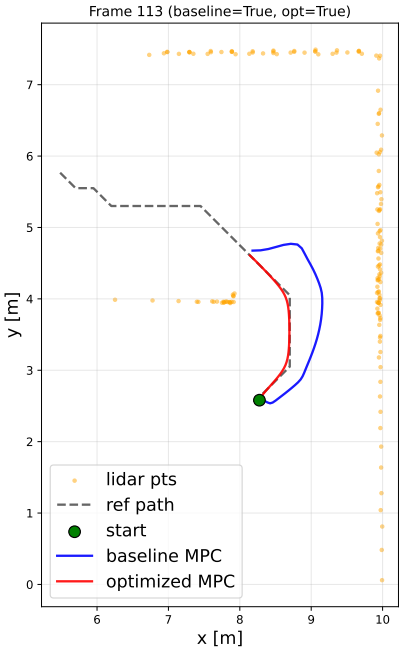

Figure 23: One frame replayed from the recorded office run: $A^\star$ reference (grey), baseline MPC prediction (blue) and tuned prediction (orange) against the LiDAR returns. The tuned controller stays on the reference where the baseline is pushed off it by the wide barrier.

5.16. Summary of design guidelines

Table 16 collects the actionable outcome of the eleven campaigns.

6. Discussion and limitations

What the formulation guarantees, and what it does not. The NLP is always feasible, because the only constraints are the dynamics and the input box; this is a real robustness property for a 10 Hz loop with no fallback. But there is no terminal set, no terminal constraint and no Lyapunov argument, so nominal asymptotic stability and recursive feasibility in the sense of [3, 9] are *not* established, and the analysis of Section 3.3.4 shows that the deployed terminal weight is far from the infinite-horizon cost-to-go of even the linearised dynamics. Safety is likewise not guaranteed: the clearance constraint is a finite penalty and it is imposed only at the discrete nodes. A formulation with a discrete-time control barrier function constraint [1], or with a soft constraint plus explicitly bounded slack, would recover a certificate; both make the NLP harder and can render it infeasible precisely when the robot is already too close to an obstacle, which is why the deployed system prefers the penalty. Quantifying that trade-off is the most valuable extension of this study.

The weakest link is not the optimizer. Every campaign in Section 5 solved its NLP successfully — zero solver failures in more than 20 000 solves — and the one systematic failure of the whole system is a *modelling* failure: the greedy rolling-horizon target rule (17), which discards the topology of the free space and, combined with the volatile grid and the non-negativity of v_x, produces the livelock of Section 5.3. The deployed stack works around it with a decaying persistent map and a topological graph. A cleaner formulation would put the local target inside the optimization — for instance a receding-horizon target chosen to minimise a value

Table 16: Design guidelines extracted from the numerical study.

Parameter	Deployed / suggested	Evidence
Horizon N	50 / **20**	Cost linear at 0.52 ms/step; performance saturates at $N \approx 20$ in both worlds (§5.7).
Sampling Δt	0.1 s / keep	95% duty cycle at 0.05 s; barrier enforced only every 20 cm at 0.4 s (§5.9).
Barrier height W_{obs}	4.83 / keep	Cliff at $W_{\text{obs}} \approx 50$: above it the robot stalls; the baseline's 500 was on the wrong side (§5.10).
Steepness α_{obs}	13.4 m^{-1} / keep	Monotone clearance-vs-progress trade-off; flat barriers ($\alpha_{\text{obs}} \leq 2$) stall and cost more iterations (§5.10).
Barrier points M	12 / **8**	0.237 ms per point, identical trajectory and iteration count from $M = 4$ to 30; large M inflates the worst case (§5.12).
Effort scaling R	$\mu = 1$ / keep	Knee of a 20× smoothness vs 50% accuracy Pareto front (§5.11).
Rate penalty R_{jerk}	2.68 / not critical	Weak knob; 20% smoothing over four decades (§5.11).
Terminal ρ	184.8 / not critical	Flat closed loop; mild conditioning gain up to $\rho \approx 100$ (§5.13).
Warm start	on / keep	−38.7% computation at an identical trajectory (§5.8).
Integral action	absent / **add**	Persistent 6 cm offset under a 0.2 m/s input disturbance (§5.14).
Planner memory	on / **required**	Without it the rolling-horizon rule livelocks in a two-room world (§5.3).
Weight tuning	per-axis GP, sequential	67 rollouts suffice; sweep in order of influence and carry each optimum forward — the naive arrangement leaves 10/11 curves uninformative (§5.4).
Weights worth tuning	$r_{\text{obs}}, \alpha_{\text{obs}}$ (then R)	$\Delta\mathcal{J} \approx 13$ against ≤ 0.11 for the other nine; six weights fall below the significance floor (§5.6).

function learned or computed on the graph — rather than fixing it by a ray intersection before the NLP is even built.

Limits of the tuning. The per-axis scheme buys its sample efficiency with an assumption of additive separability, and that assumption is not exactly true: Section 5.10 exhibits the $(W_{\text{obs}}, r_{\text{obs}})$ interaction it cannot represent. Sweeping the dominant coordinate first and carrying each optimum forward contains the damage, and the assembled vector is verified by evaluation rather than assumed to be optimal, but a genuine two-parameter interaction can only be found by a joint search over the pair — which, restricted to the two or three coordinates the sweeps identify as decisive, would be affordable and is the natural next step. Two further limits are worth stating. The tuning objective (34) was evaluated in a single world, so the weights are optimal for a pillar field and merely good elsewhere; a weighted sum over several worlds would cost proportionally more rollouts. And five of the eleven axes terminate on a box bound (Table 4), so the intervals are binding and the reported optima are, on those axes, statements about the box as much as about the objective.

Scope of the numbers. All timings were obtained on a laptop-class x86 CPU. The onboard Jetson Orin Nano is roughly 3–4× slower, so the deployed $N = 50$, $\Delta t = 0.1$ s configuration sits much closer to its deadline on the robot than Fig. 14 suggests — which is precisely why the $N \to 20$ recommendation matters. The plant used in the harness is the MPC's own model perturbed in Section 5.14; it contains no leg dynamics, no terrain and no LiDAR occlusion, so the absolute success rates are optimistic while the relative comparisons, which is what the study is about, are not affected.

7. Conclusions

This report analysed a deployed map-free navigation stack as what it is numerically: two optimization programs solved at every cycle — an exact $A^\star$ search on a Gaussian occupancy grid and a 456-variable nonlinear program with actuator-lag dynamics and a smooth obstacle barrier — plus one black-box program solved offline to choose the eleven cost weights of the second.

The main quantitative findings are the following. The problem is genuinely real-time: the interior-point solve

costs 0.52 ms per horizon step, is linear in N, needs 7–9 iterations per cycle, and respects a 100 ms budget at every cycle except the first, which pays the one-off symbolic construction. The shift-and-append warm start removes 38.7% of that cost while reproducing the same trajectory to 10^{-4} m. Closed-loop performance saturates at $N \approx 20$, so the deployed $N = 50$ costs 2.2× more computation than the task requires. Tuning the eleven weights with one one-dimensional Gaussian process per weight — 67 closed-loop rollouts in total — changes the outcome qualitatively rather than quantitatively: the hand-tuned baseline never reaches the goal in any of the three test worlds, the tuned configuration does, while also halving the mean solve time and cutting the iteration count by up to 58%. The eleven measured curves and an independent closed-loop ablation identify the same cause: two weights out of eleven decide the outcome, and the hand-tuned barrier they parametrise was wide enough for its repulsive field, rather than the tracking cost, to dictate the solution. Sweeping the axes in order of influence and carrying each optimum forward is what makes so small a budget sufficient — run naively about a single base point, the same sweeps leave ten of eleven curves measuring differences between failures.

Two negative results are as informative as the positive ones. First, the deployed heuristic terminal cost is 40× heavier than the exact infinite-horizon cost-to-go on position, ignores the terminal velocity states entirely, and the closed loop is nonetheless indifferent to it over three decades — with a 5 s horizon and a reference regenerated every second, the terminal weight regularises but does not stabilise. Second, and more consequentially, the only systematic failure of the system is not in the optimization at all: the greedy rolling-horizon target rule livelocks in a two-room world where every individual NLP is solved correctly. Posing the right problem remains harder than solving it well.

The natural continuations follow directly from the analysis: add integral action or a disturbance estimator to remove the measured steady-state offset; re-run the outer optimization on a widened box and against a closed-loop, mission-level objective, which the harness built for this report makes affordable; reduce N to 20 and M to 8 to recover the resulting computational headroom on the onboard computer; and replace the ray-based local target with one that respects the topology of the free space, so that the quality of the solution matches the quality of the solver.

Reproducibility

Every figure and table in Section 5 is generated by an offline harness that imports the production planner and tracker from the project repository, runs the parametric campaigns in closed loop (>20 000 IPOPT solves in total) and serialises the logged signals; the figures are then produced from those logs. The tuning curves of Fig. 11 come from 67 additional closed-loop rollouts, six per axis plus one confirmation of the assembled vector, and the hardware figures from recorded ROS 2 bags of the physical robot. The deployed parameter vector θ_{dep} of Table 4 is the one shipped in `src/a_star_mpc_planner/ config/planner_params.yaml`. Animated versions of Fig. 12, showing the three configurations running the same mission side by side with live readouts of every quantity quoted in Section 5.5, accompany this report as `Videos/compare_pillar_field.mp4` and `Videos/compare_s_corridor.mp4`. Solve times in those recordings are wall-clock and therefore load-dependent: their absolute values differ from Table 5 by the machine load at recording time, while the ratio between configurations is stable to within 1%.

References

[1] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *18th European Control Conference (ECC)*, pages 3420–3431, 2019.

[2] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi – a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

[3] Hong Chen and Frank Allgöwer. A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica*, 34(10):1205–1217, 1998.

[4] Moritz Diehl, Rolf Findeisen, Frank Allgöwer, Hans Georg Bock, and Johannes P. Schlöder. Nominal stability of real-time iteration scheme for nonlinear model predictive control. In *IEE Proceedings – Control Theory and Applications*, volume 152, pages 296–308, 2005.

[5] Dong Jin Hyun, Sangok Seok, Jongwoo Lee, and Sangbae Kim. High speed trot-running: implementation of a hierarchical controller using proprioceptive impedance control on the MIT Cheetah. *The International Journal of Robotics Research*, 33(11):1417–1445, 2014.

[6] Juan Miguel Jimeno. CHAMP: Hierarchical controller for highly dynamic quadrupedal locomotion. `https://github.com/chvmp/champ`, 2020.

[7] Jongwoo Lee. Hierarchical controller for highly dynamic locomotion utilizing pattern modulation and impedance control: implementation on the MIT Cheetah robot. M.Sc. thesis, Massachusetts Institute of Technology, Department of Mechanical Engineering, Cambridge, MA, USA, 2013. `https://dspace.mit.edu/handle/1721.1/85490`.

[8] Jialong Li, Xuxin Cheng, Tianshu Huang, Shiqi Yang, Ri-Zhao Qiu, and Xiaolong Wang. AMO: Adaptive motion optimization for hyper-dexterous humanoid whole-body control. In *Proceedings of Robotics: Science and Systems (RSS)*, 2025. `https://arxiv.org/abs/2505.03738`.

[9] David Q. Mayne, James B. Rawlings, Christopher V. Rao, and Pierre O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[10] Lorenzo Ortolani, Gabriel Voss, Gabriele Beltrami, Francesco Dorati, and Tommaso Felice Banfi. Bayesian optimization for learning nonlinear MPC in autonomous agent navigation. Talos Robotics AI, Milan, Italy. Companion paper of the `Go2_navigation` project, 2026.

[11] Marc H. Raibert. *Legged Robots That Balance*. MIT Press, Cambridge, MA, USA, 1986.

[12] Carl E. Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[13] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017.

[14] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022. `https://proceedings.mlr.press/v164/rudin22a.html`.

[15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. `https://arxiv.org/abs/1707.06347`.

[16] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[17] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006.